Relatório Completo do Estudo 3

Marcos Belo Jefferson Amorim

8 de novembro de 2026

1 Estudo 3 — Filtros Digitais: Análise, Projeto e Aplicações

Instituição: Instituto Federal da Paraíba — IFPB
Disciplina: Processamento Digital de Sinais
Professor: Moacy Pereira da Silva
Equipe: Marcos Vinicius Belo da Silva e Jefferson Amorim da Costa

1.1 Introdução

O estudo aplica os fundamentos dos estudos anteriores ao projeto de filtros FIR e IIR. Reúne os 14 tópicos do guia e um tratamento digital de sensor com validação quantitativa. Os resultados computacionais são reproduzíveis pelos notebooks incluídos. Os sinais dos projetos são sintéticos.

2 Conceito de Filtragem Digital

---

3 Resumo conceitual

Filtrar é modificar seletivamente componentes de um sinal. Passa-baixas suaviza sensores, passa-altas remove deriva, passa-faixa seleciona uma banda de vibração e rejeita-faixa remove interferência localizada.

---

4 Formulação matemática

$$y[n] = x[n] * h[n], \qquad Y(e^{j\omega}) = X(e^{j\omega})H(e^{j\omega}). \tag{1}$$

Um filtro ideal tem magnitude 1 na banda preservada e 0 na banda removida; transições abruptas exigem resposta ao impulso infinita e, em geral, não causal.

---

5 Exemplo

Represente os quatro tipos ideais a fs = 1000 Hz, com limites de 100 e 200 Hz.

---

6 Simulação

Represente os quatro tipos ideais a fs = 1000 Hz, com limites de 100 e 200 Hz.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

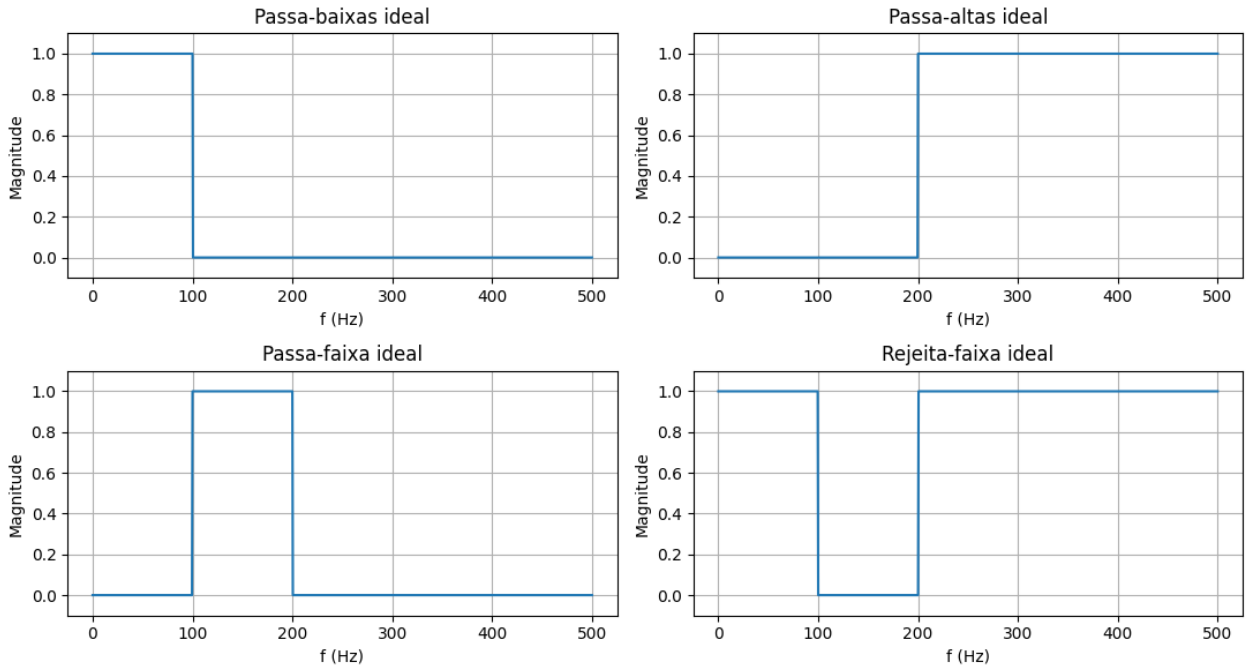
```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

f=np.linspace(0,500,1001)
curvas={'Passa -baixas':f<=100,'Passa -altas':f>=200,'Passa -faixa':(f>=100)&(f<=200),'Rejeita -faixa':(f
     <100)|(f>200)}
fig,ax=plt.subplots(2,2,figsize=(11,6))
for a,(nome,H) in zip(ax.flat,curvas.items()):
    a.plot(f,H.astype(float)); a.set(title=nome+' ideal',xlabel='f (Hz)',ylabel='Magnitude',ylim=( -.1,1.1))
plt.tight_layout(); plt.show()
```



---

7 Resultados

As quatro máscaras mostram as bandas preservadas e rejeitadas. A resposta ideal estabelece um objetivo de projeto, sem fornecer uma implementação causal finita.

8 Interpretação e Discussão

Filtros realizáveis têm transição, ripple, atraso e atenuação finita. A seleção depende da faixa que contém informação útil e daquela que contém interferência.

9 Conclusões

As quatro máscaras mostram as bandas preservadas e rejeitadas. A resposta ideal estabelece um objetivo de projeto, sem fornecer uma implementação causal finita.

10 Resposta em Frequência

---

11 Resumo conceitual

A resposta em frequência especifica o ganho e a fase aplicados a cada senoide. A banda passante conserva componentes dentro de tolerâncias, e a banda de rejeição as atenua.

---

12 Formulação matemática

$$H(z) = \frac{1/3(1 + z^{-1} + z^{-2})}{1}, \quad h[n] = \tfrac{1}{3}\{1, 1, 1\}, \quad H(e^{j\omega}) = \tfrac{1}{3}(1 + 2\cos\omega)e^{-j\omega}. \tag{2}$$

Ripple é a variação de ganho na banda; seletividade descreve a rapidez da transição. A frequência de corte precisa de uma convenção, frequentemente -3 dB.

---

13 Exemplo

Analise a média móvel de três amostras a fs = 1000 Hz. Identifique ganho em DC, primeiro nulo e fase.

---

14 Simulação

Analise a média móvel de três amostras a fs = 1000 Hz. Identifique ganho em DC, primeiro nulo e fase.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

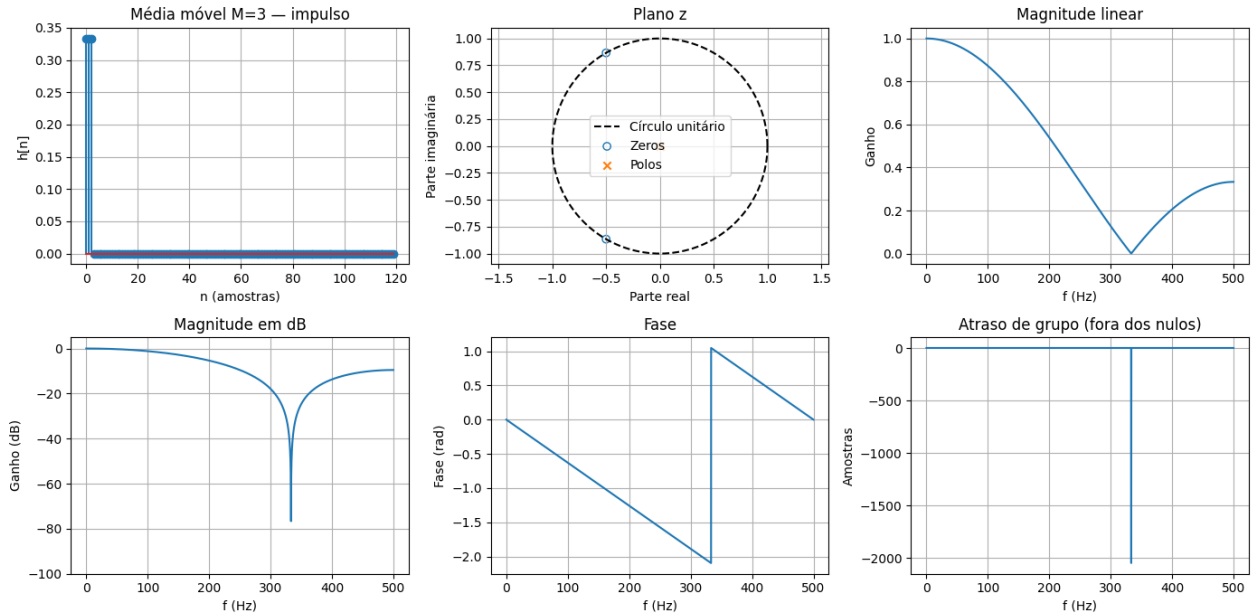
def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
```

```
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

analisar(np.ones(3)/3,fs=1000,nome='Média móvel M=3')
print('Ganho DC = 1; primeiro nulo =',1000/3,'Hz')
```




```
b = [0.33333333 0.33333333 0.33333333]
a = [1]
Polos: [] Estável causal: True
Ganho DC = 1; primeiro nulo = 333.3333333333333 Hz
```

---

15 Resultados

O ganho em DC é 1 e o primeiro nulo ocorre em 333,33 Hz. A fase é linear por trechos, com saltos de π associados a mudanças de sinal da parcela real.

16 Interpretação e Discussão

Um passa-baixas real não possui uma única fronteira perfeita: a transição separa as bandas especificadas. O nulo de uma média móvel não é a mesma coisa que sua frequência de corte.

17 Conclusões

O ganho em DC é 1 e o primeiro nulo ocorre em 333,33 Hz. A fase é linear por trechos, com saltos de π associados a mudanças de sinal da parcela real.

18 Magnitude em Decibéis

---

19 Resumo conceitual

A escala em decibéis expressa razões de forma logarítmica e facilita comparar ganhos muito diferentes. Relações de amplitude usam fator 20; razões de potência usam fator 10.

---

20 Formulação matemática

$$G_{dB} = 20 \log_{10} |H|, \quad |H| = 10^{G_{dB}/20}. \tag{3}$$

Sob a mesma impedância, a potência é proporcional ao quadrado da amplitude.

---

21 Exemplo

Converta 0, -3, -20 e -40 dB e represente a mesma média móvel em escala linear e logarítmica.

---

22 Simulação

Converta 0, -3, -20 e -40 dB e represente a mesma média móvel em escala linear e logarítmica. Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

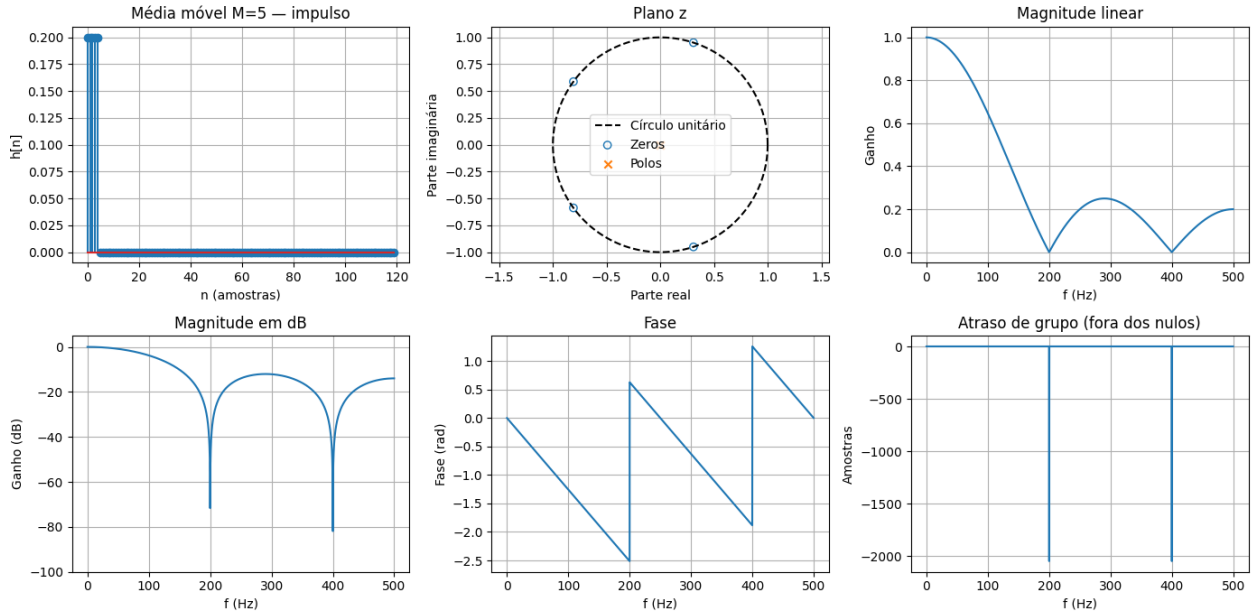
def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
```

```
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

g=np.array([0, -3, -20, -40]); display(pd.DataFrame({'Ganho (dB)':g,'Razão de amplitude':10.**(g/20),'Razão
     de potência':10.**(g/10)}))
analisar(np.ones(5)/5,fs=1000,nome='Média móvel M=5')
```

	Ganho (dB)	Razão de amplitude	Razão de potência
0	0	1.000000	1.000000
1	-3	0.707946	0.501187
2	-20	0.100000	0.010000
3	-40	0.010000	0.000100




```
b = [0.2 0.2 0.2 0.2 0.2]
a = [1]
Polos: [] Estável causal: True
```

---

23 Resultados

As razões de amplitude são 1; 0,70795; 0,1 e 0,01. -3 dB corresponde aproximadamente à metade da potência.

24 Interpretação e Discussão

Ganho negativo em dB significa atenuação, e não amplitude negativa. Nulos exatos corresponderiam a -∞ dB; o código aplica um piso apenas para permitir sua representação gráfica.

25 Conclusões

As razões de amplitude são 1; 0,70795; 0,1 e 0,01. -3 dB corresponde aproximadamente à metade da potência.

26 Filtros FIR

---

27 Resumo conceitual

Filtros FIR possuem resposta ao impulso finita e podem ser implementados por uma soma de entradas atrasadas. Coeficientes finitos garantem estabilidade BIBO; simetria permite fase linear.

---

28 Formulação matemática

$$y[n] = \sum_{k=0}^{M-1} b_k x[n-k], \quad H(z) = \sum_{k=0}^{M-1} b_k z^{-k}. \tag{4}$$

Na média móvel $b_k = 1/M$, a ordem é $M-1$, o atraso é $(M-1)/2$ amostras e o primeiro nulo ocorre em f_s/M.

---

29 Exemplo

Compare M = 3, 5 e 10. Observe os coeficientes, polos na origem e zeros sobre o círculo unitário, exceto z = 1.

---

30 Simulação

Compare M = 3, 5 e 10. Observe os coeficientes, polos na origem e zeros sobre o círculo unitário, exceto z = 1.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

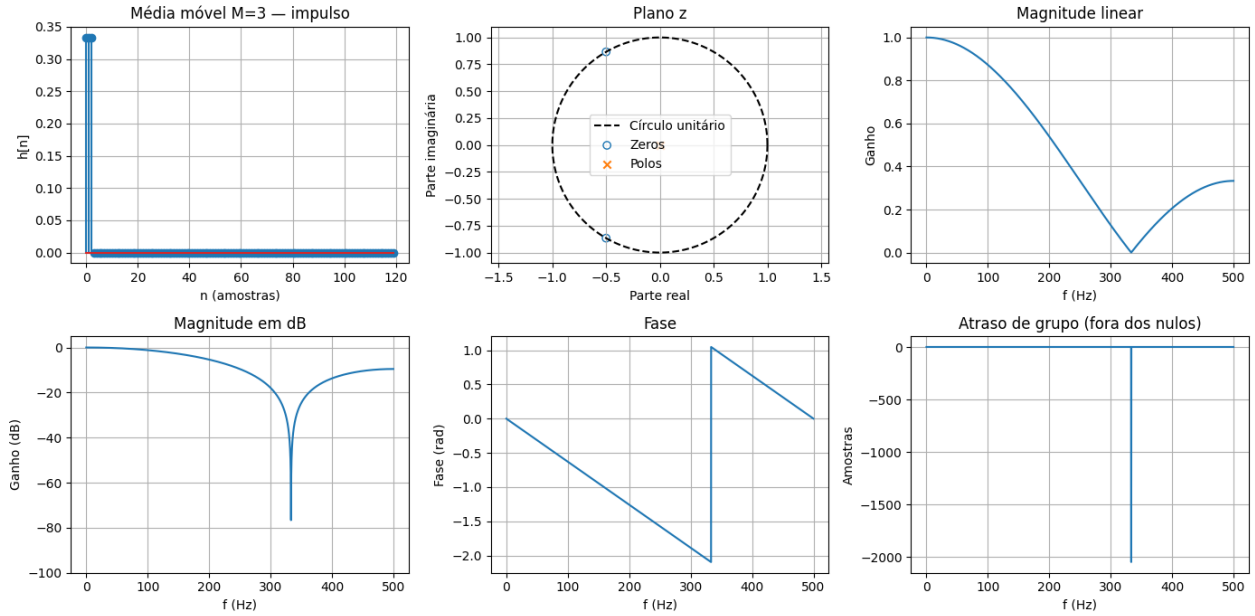
def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
```

```
        A[ -1] /= 2
    return f, A

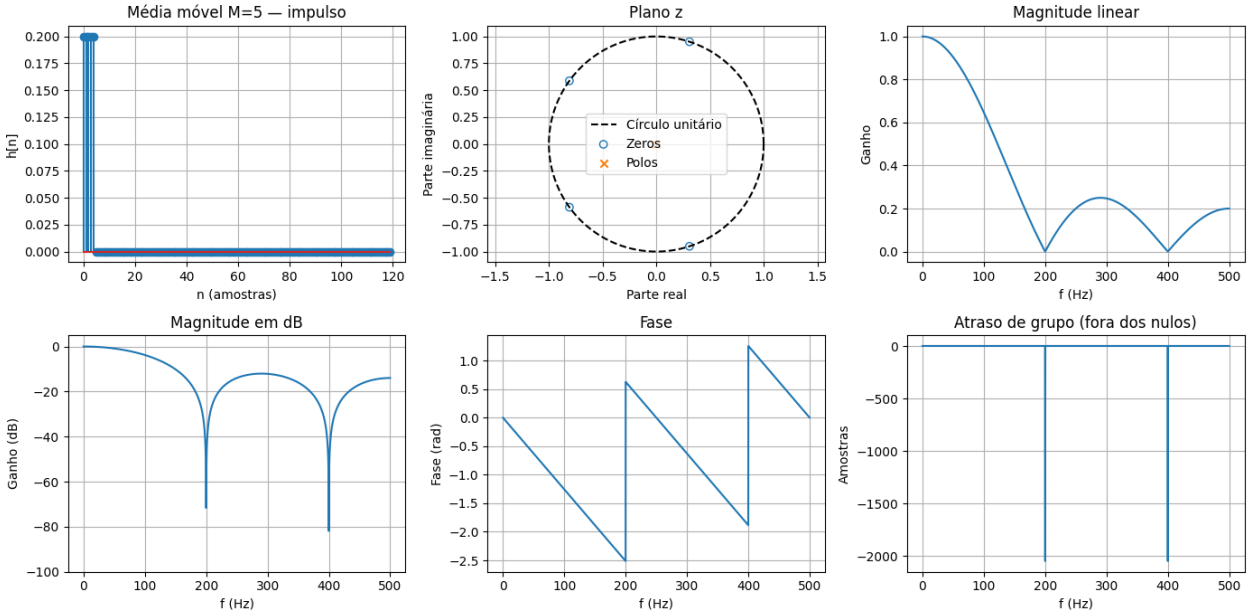
def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

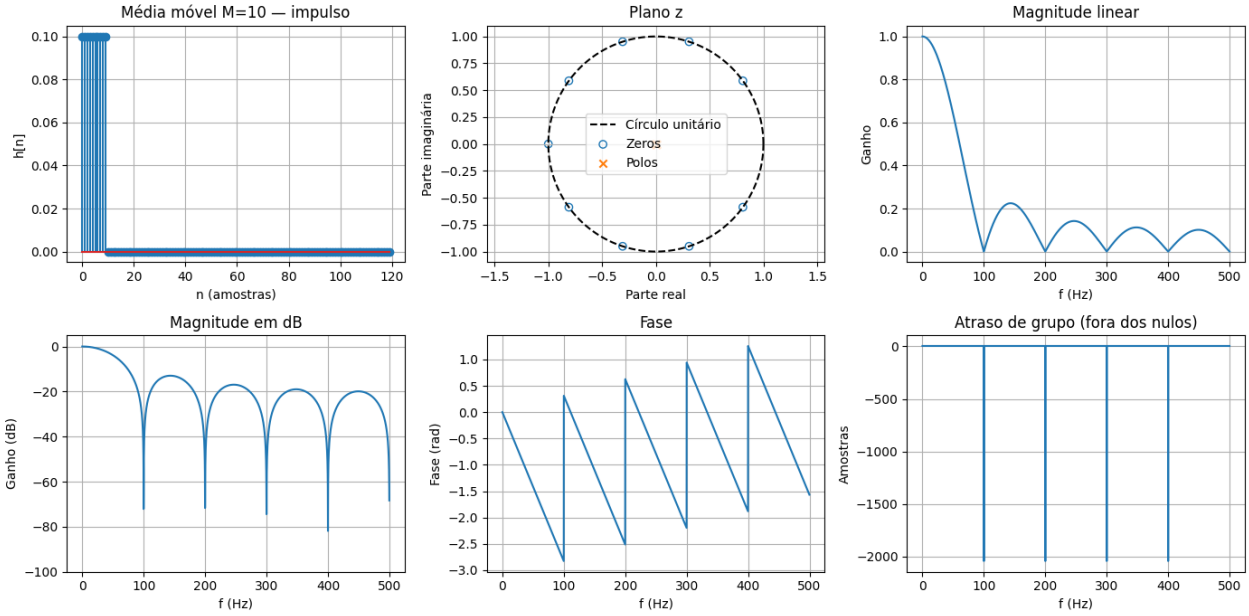
for M in [3,5,10]:
    analisar(np.ones(M)/M,fs=1000,nome=f'Média móvel M={M}')
    print('Ordem:',M -1,'Atraso:',(M -1)/2,'amostras; primeiro nulo:',1000/M,'Hz')
```




```
b = [0.33333333 0.33333333 0.33333333]
a = [1]
Polos: [] Estável causal: True
Ordem: 2 Atraso: 1.0 amostras; primeiro nulo: 333.3333333333333 Hz
```




```
b = [0.2 0.2 0.2 0.2 0.2]
a = [1]
Polos: [] Estável causal: True
Ordem: 4 Atraso: 2.0 amostras; primeiro nulo: 200.0 Hz
```




```
b = [0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1]
a = [1]
Polos: [] Estável causal: True
Ordem: 9 Atraso: 4.5 amostras; primeiro nulo: 100.0 Hz
```

---

31 Resultados

Os primeiros nulos são 333,33; 200 e 100 Hz. Os atrasos são 1; 2 e 4,5 amostras. A soma dos coeficientes é 1 em todos os casos.

32 Interpretação e Discussão

O comprimento maior estreita o lóbulo principal, mas a média móvel tem rejeição limitada nos lóbulos laterais. Os polos na origem aparecem ao escrever H como uma razão de polinômios em z; eles representam atrasos e não instabilidade.

33 Conclusões

Os primeiros nulos são 333,33; 200 e 100 Hz. Os atrasos são 1; 2 e 4,5 amostras. A soma dos coeficientes é 1 em todos os casos.

34 Filtros IIR

---

35 Resumo conceitual

Filtros IIR usam realimentação para produzir uma resposta ao impulso teoricamente infinita. Poucos coeficientes podem gerar memória longa, mas a estabilidade depende dos polos.

---

36 Formulação matemática

$$y[n] = 0{,}8y[n-1] + 0{,}2x[n], \quad H(z) = \frac{0{,}2}{1 - 0{,}8z^{-1}}, \quad h[n] = 0{,}2(0{,}8)^n u[n]. \tag{5}$$

Há um polo em 0,8 e um zero em 0 ao escrever $H(z) = 0{,}2z/(z - 0{,}8)$.

---

37 Exemplo

Analise o filtro recursivo de primeira ordem, com repouso inicial e fs = 1000 Hz.

---

38 Simulação

Analise o filtro recursivo de primeira ordem, com repouso inicial e fs = 1000 Hz.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

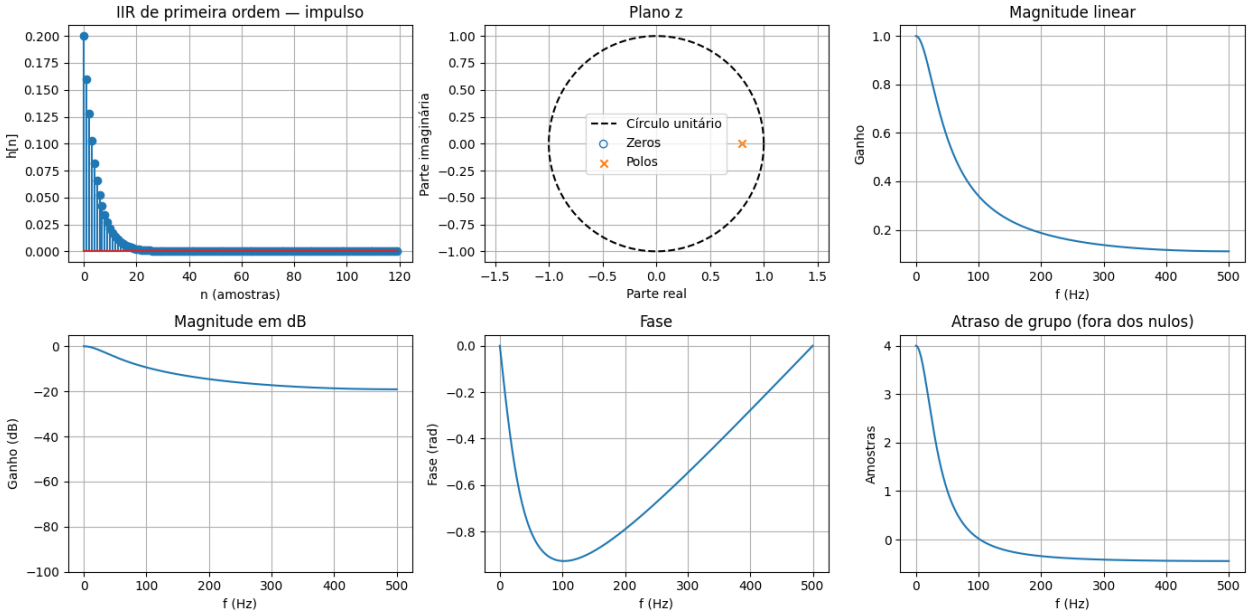
def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
```

```
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

analisar([.2],[1, -.8],nome='IIR de primeira ordem')
print('Ganho DC:',.2/(1 -.8))
```




```
b = [0.2]
a = [ 1.  -0.8]
Polos: [0.8] Estável causal: True
Ganho DC: 1.0000000000000002
```

39 Resultados

A resposta decai exponencialmente sem se tornar exatamente nula. O ganho em DC é 1, o polo está dentro do círculo e a fase é não linear.

40 Interpretação e Discussão

A primeira ordem exige duas multiplicações por amostra na forma direta. A economia de coeficientes vem acompanhada de realimentação e maior atenção a arredondamento, saturação e estabilidade.

41 Conclusões

A resposta decai exponencialmente sem se tornar exatamente nula. O ganho em DC é 1, o polo está dentro do círculo e a fase é não linear.

42 Comparação FIR e IIR

---

43 Resumo conceitual

FIR e IIR podem cumprir objetivos espectrais semelhantes com compromissos diferentes de ordem, fase e custo. A escolha depende também da latência e da precisão numérica.

---

44 Formulação matemática

$$H_{FIR}(z) = \sum_{k=0}^{50} b_k z^{-k}, \qquad H_{IIR}(z) = B(z)/A(z). \tag{6}$$

Um FIR simétrico de 51 coeficientes tem atraso de 25 amostras. Na forma direta, o custo aproximado é $N_b + N_a - 1$ multiplicações por amostra, com $a_0 = 1$ não multiplicado.

---

45 Exemplo

Compare um FIR Hamming de 51 coeficientes com um Butterworth IIR de ordem 4, ambos passa-baixas com parâmetro de corte de 100 Hz e fs = 1000 Hz.

---

46 Simulação

Compare um FIR Hamming de 51 coeficientes com um Butterworth IIR de ordem 4, ambos passa-baixas com parâmetro de corte de 100 Hz e fs = 1000 Hz.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

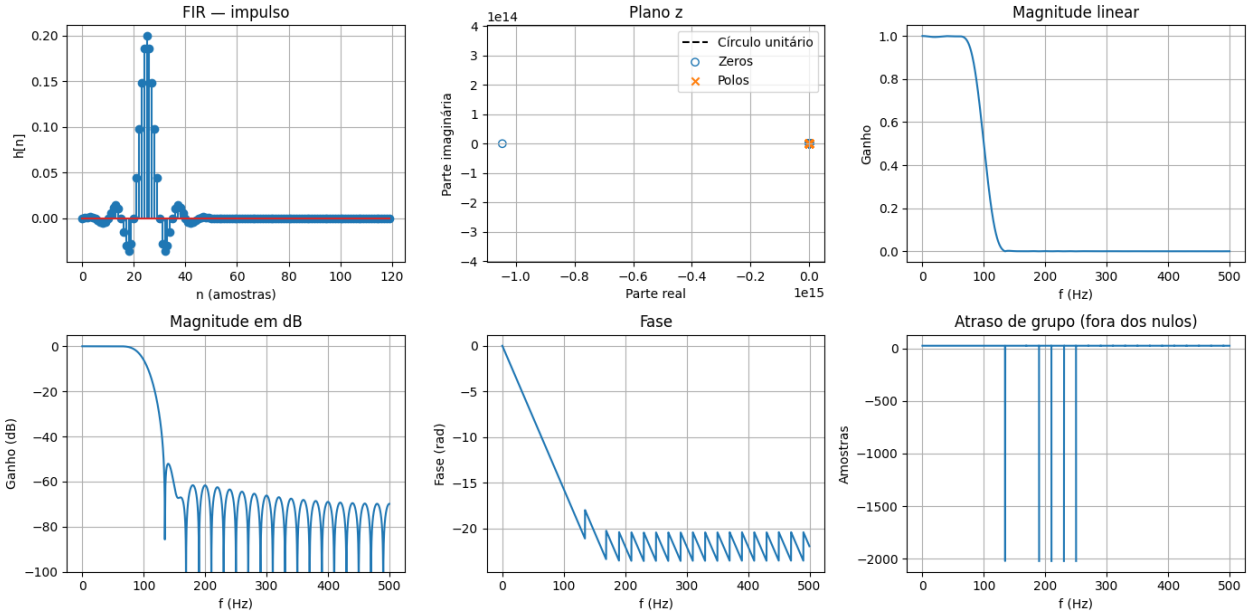
```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

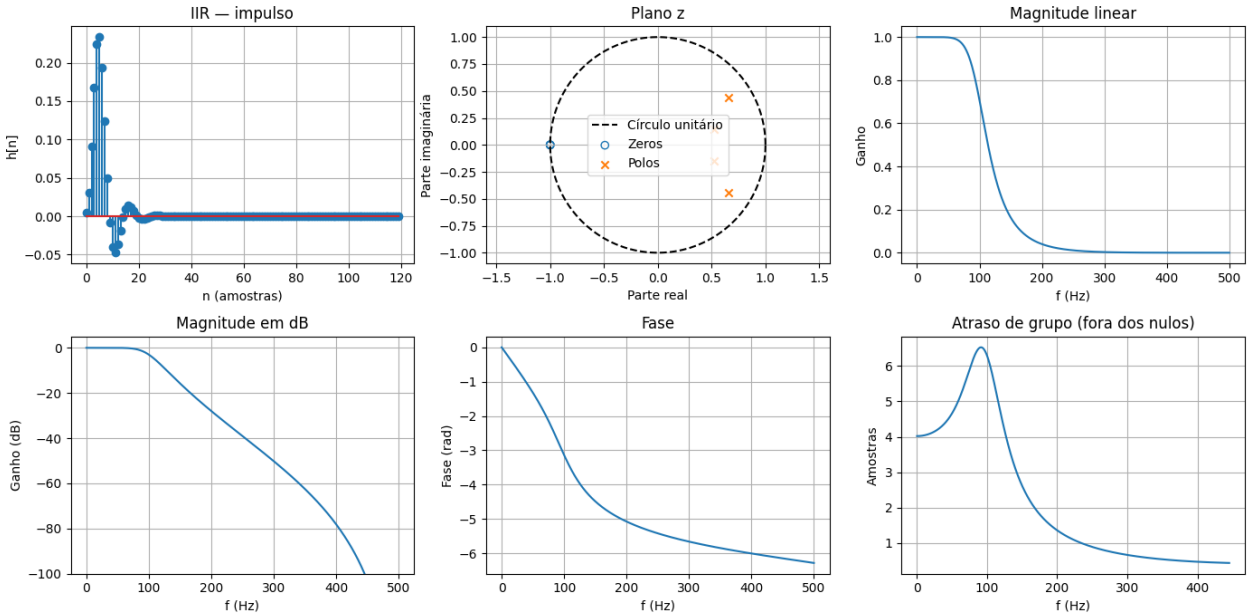
def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

fs=1000; bfir=signal.firwin(51,100,fs=fs); biir,aiir=signal.butter(4,100,fs=fs)
for nome,b,a in [('FIR',bfir,[1]),('IIR',biir,aiir)]:
    analisar(b,a,fs,nome)
    print(nome,'ordem =',max(len(b),len(a)) -1,'coeficientes armazenados =',len(b)+len(a) -1,'multiplicações
     /amostra (forma direta) =',len(b)+len(a) -1)
```




```
b = [ 6.22233219e -19  6.50395537e -04  1.24025458e -03  1.54168857e -03
  1.21680726e -03 -1.30554005e -18 -2.01071740e -03 -4.14071058e -03
 -5.21455172e -03 -4.01474662e -03  3.09446055e -18  6.05036949e -03
  1.18742206e -02  1.43176216e -02  1.06257168e -02 -5.30568790e -18
 -1.52613457e -02 -2.96911186e -02 -3.59490768e -02 -2.72307378e -02
  7.09460841e -18  4.40087761e -02  9.74197266e -02  1.48825236e -01
  1.85978564e -01  1.99527255e -01  1.85978564e -01  1.48825236e -01
  9.74197266e -02  4.40087761e -02  7.09460841e -18 -2.72307378e -02
 -3.59490768e -02 -2.96911186e -02 -1.52613457e -02 -5.30568790e -18
  1.06257168e -02  1.43176216e -02  1.18742206e -02  6.05036949e -03
  3.09446055e -18 -4.01474662e -03 -5.21455172e -03 -4.14071058e -03
 -2.01071740e -03 -1.30554005e -18  1.21680726e -03 -1.54168857e -03
  1.24025458e -03  6.50395537e -04  6.22233219e -19]
a = [1]
Polos: [] Estável causal: True
FIR ordem = 50 coeficientes armazenados = 51 multiplicações/amostra (forma direta) = 51
```




```
b = [0.00482434 0.01929737 0.02894606 0.01929737 0.00482434]
a = [ 1.         -2.36951301  2.31398841 -1.05466541  0.18737949]
Polos: [0.66045672+0.44332349j 0.66045672 -0.44332349j 0.52429979+0.1457741j
 0.52429979 -0.1457741j ] Estável causal: True
IIR ordem = 4 coeficientes armazenados = 9 multiplicações/amostra (forma direta) = 9
```

---

47 Resultados

O FIR tem ordem 50 e 51 multiplicações por amostra na implementação direta; o IIR tem ordem 4 e 9. O FIR apresenta atraso constante na banda e o IIR apresenta atraso dependente da frequência.

48 Interpretação e Discussão

A comparação é ilustrativa: o parâmetro de corte em firwin corresponde aproximadamente a -6 dB e em Butterworth a -3 dB. Igualar esse parâmetro não iguala as especificações; o mini projeto compara soluções verificadas contra as mesmas bandas e tolerâncias.

49 Conclusões

O FIR tem ordem 50 e 51 multiplicações por amostra na implementação direta; o IIR tem ordem 4 e 9. O FIR apresenta atraso constante na banda e o IIR apresenta atraso dependente da frequência.

50 Polos, Zeros e Resposta em Frequência

---

51 Resumo conceitual

A magnitude depende das distâncias do círculo unitário aos zeros e polos. Um zero sobre esse círculo produz um nulo; um polo próximo produz uma região de maior ganho e memória longa.

---

52 Formulação matemática

$$|H(e^{j\omega})| = |K| \frac{\prod_i |e^{j\omega} - z_i|}{\prod_k |e^{j\omega} - p_k|}. \tag{7}$$

Para um rejeita-faixa em ω_0, o numerador pode ser $1 - 2\cos\omega_0 z^{-1} + z^{-2}$; polos com raio r controlam a largura.

---

53 Exemplo

Construa filtros com zeros em $e^{\pm j0{,}4\pi}$ e polos no mesmo ângulo com raios 0,5 e 0,95. Normalize o ganho em DC.

---

54 Simulação

Construa filtros com zeros em $e^{\pm j0,4\pi}$ e polos no mesmo ângulo com raios 0,5 e 0,95. Normalize o ganho em DC.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

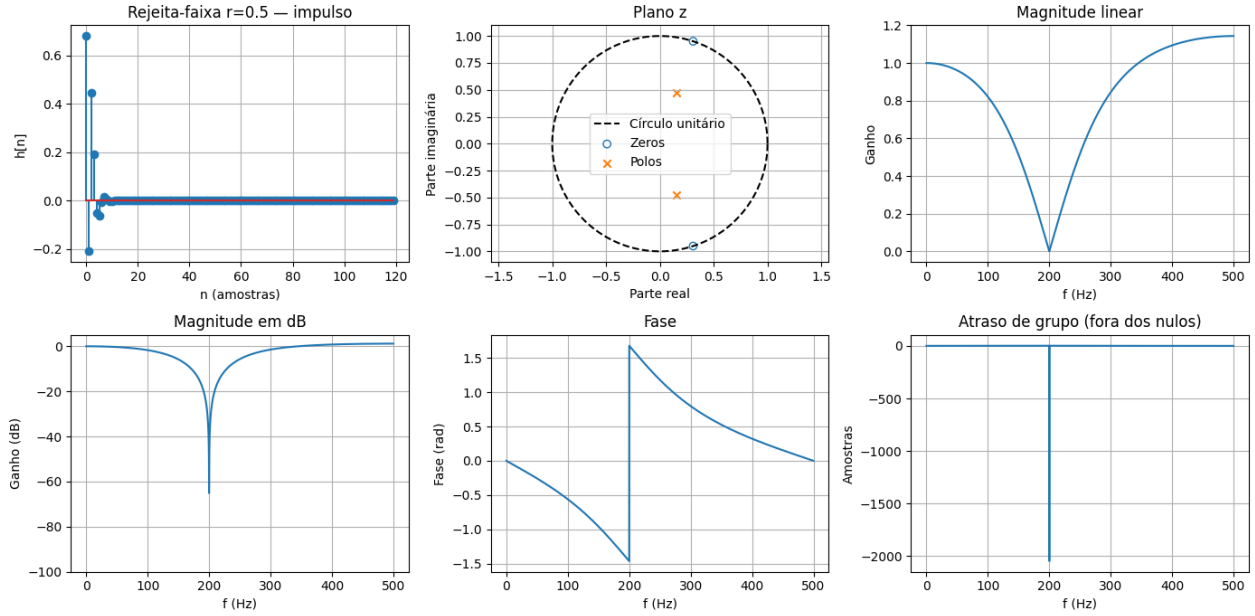
```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

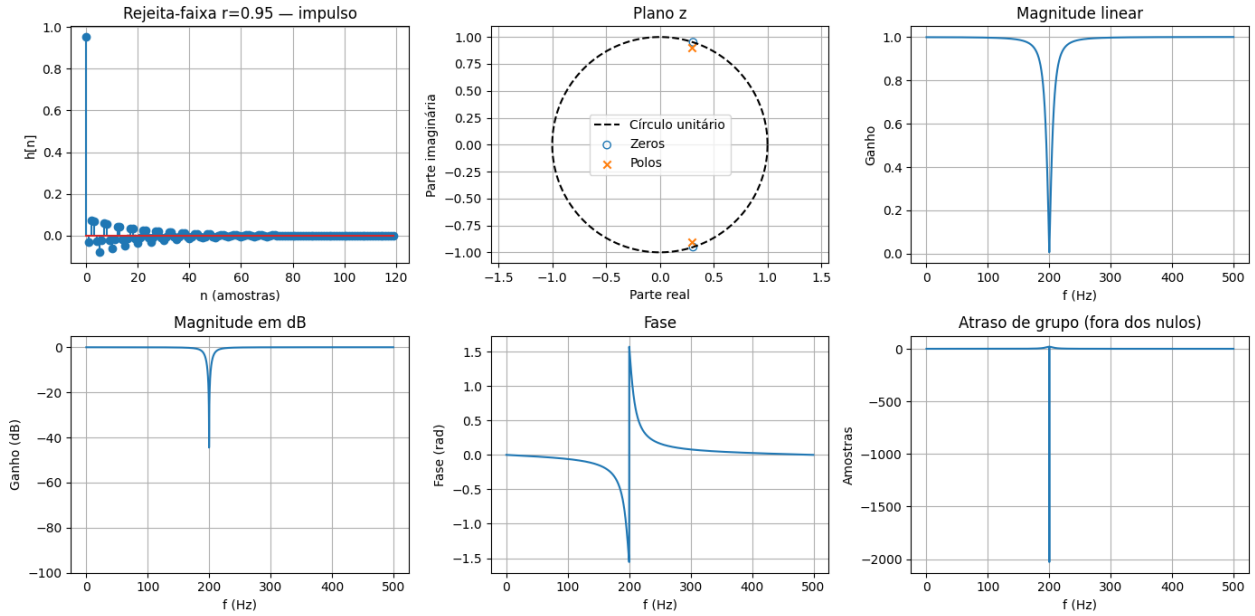
def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

w0=.4*np.pi
for r in [.5,.95]:
    b=np.array([1, -2*np.cos(w0),1]); a=np.array([1, -2*r*np.cos(w0),r*r]); b*=a.sum()/b.sum()
    analisar(b,a,1000,f'Rejeita -faixa r={r}')
```




```
b = [ 0.6809017  -0.42082039  0.6809017 ]
a = [ 1.         -0.30901699  0.25      ]
Polos: [0.1545085+0.47552826j 0.1545085 -0.47552826j] Estável causal: True
```




```
b = [ 0.95180902 -0.58825032  0.95180902]
a = [ 1.         -0.58713229  0.9025    ]
Polos: [0.29356614+0.90350369j 0.29356614 -0.90350369j] Estável causal: True
```

---

55 Resultados

O nulo ocorre em 200 Hz. Com raio 0,95, a rejeição fica mais estreita e a resposta ao impulso demora mais para decair.

56 Interpretação e Discussão

A geometria ajuda a prever a forma do espectro, mas o ganho K precisa ser considerado para comparar amplitudes absolutas. Polos muito próximos do círculo também aumentam a

sensibilidade à precisão dos coeficientes.

57 Conclusões

O nulo ocorre em 200 Hz. Com raio 0,95, a rejeição fica mais estreita e a resposta ao impulso demora mais para decair.

58 Estabilidade dos Filtros IIR

---

59 Resumo conceitual

A estabilidade BIBO de um filtro causal racional exige todos os polos estritamente dentro do círculo unitário. A proximidade à borda aumenta o tempo de acomodação.

---

60 Formulação matemática

$$h[n] = p^n u[n], \qquad |p| < 1 \Rightarrow \sum_{n=0}^{\infty} |h[n]| = 1/(1-|p|). \tag{8}$$

O tempo para atingir 1% da amplitude inicial é aproximadamente $\ln(0{,}01)/\ln|p|$ amostras.

---

61 Exemplo

Compare p = 0,5; 0,9; 0,99 e 1,05 com entrada impulso.

---

62 Simulação

Compare p = 0,5; 0,9; 0,99 e 1,05 com entrada impulso.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

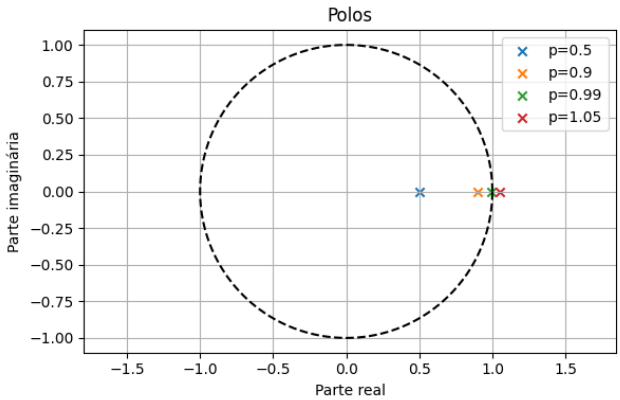
def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A
```

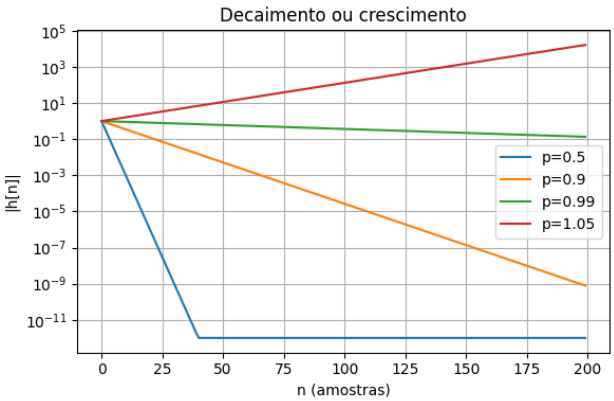
```
def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

n=np.arange(200); fig,ax=plt.subplots(1,2,figsize=(12,4)); th=np.linspace(0,2*np.pi,500); ax[0].plot(np.cos(
    th),np.sin(th),'k - -')
for p in [.5,.9,.99,1.05]:
    h=signal.lfilter([1],[1, -p],np.r_[1.,np.zeros(199)])
    ax[0].scatter(p,0,marker='x',label=f'p={p}')
    ax[1].semilogy(n,np.maximum(abs(h),1e -12),label=f'p={p}')
    print(p,'estável:',abs(p)<1,'n para 1%:',np.log(.01)/np.log(p) if p<1 else 'não converge')
ax[0].axis('equal'); ax[0].set(xlabel='Parte real',ylabel='Parte imaginária',title='Polos'); ax[1].set(
    xlabel='n (amostras)',ylabel='|h[n]|',title='Decaimento ou crescimento')
for a in ax: a.legend()
plt.tight_layout(); plt.show()
```

```
0.5 estável: True n para 1%: 6.643856189774724
0.9 estável: True n para 1%: 43.70869065356567
0.99 estável: True n para 1%: 458.2105765533884
1.05 estável: False n para 1%: não converge
```





63 Resultados

Os três primeiros casos são estáveis; 1,05 é instável. Para 0,99 são necessárias cerca de 459 amostras para atingir 1% da amplitude inicial.

64 Interpretação e Discussão

Uma resposta lenta pode parecer constante em uma janela curta. A verificação analítica dos polos deve acompanhar a simulação; um polo em 1 não é estritamente estável BIBO.

65 Conclusões

Os três primeiros casos são estáveis; 1,05 é instável. Para 0,99 são necessárias cerca de 459 amostras para atingir 1% da amplitude inicial.

66 Fase e Atraso

---

67 Resumo conceitual

A fase descreve o deslocamento angular aplicado a cada componente. Uma fase linear impõe um atraso comum; fase não linear pode alterar a forma de sinais compostos mesmo quando as amplitudes são preservadas.

---

68 Formulação matemática

$$\phi(\omega) = -\omega n_0 + \phi_0, \qquad b_k = b_{M-k} \Rightarrow n_0 = M/2. \tag{9}$$

Em regiões sem zeros, um FIR simétrico apresenta fase linear por trechos, podendo existir saltos de π.

---

69 Exemplo

Compare o FIR simétrico de ordem 50 e o IIR Butterworth de ordem 4 a fs = 1000 Hz.

---

70 Simulação

Compare o FIR simétrico de ordem 50 e o IIR Butterworth de ordem 4 a fs = 1000 Hz. Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

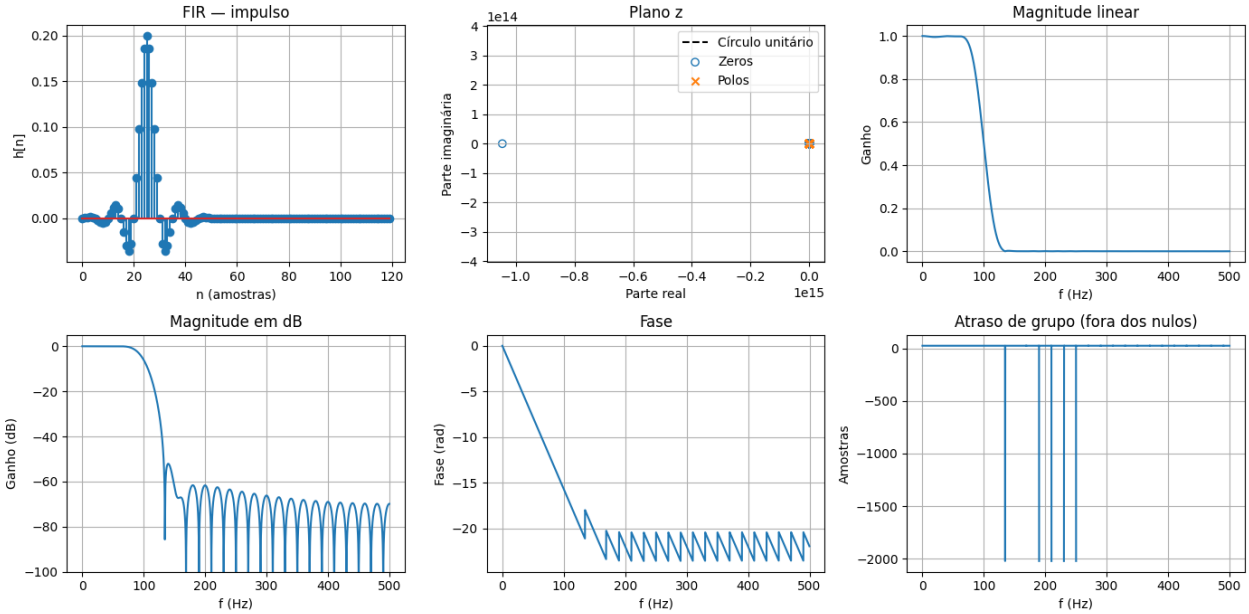
```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

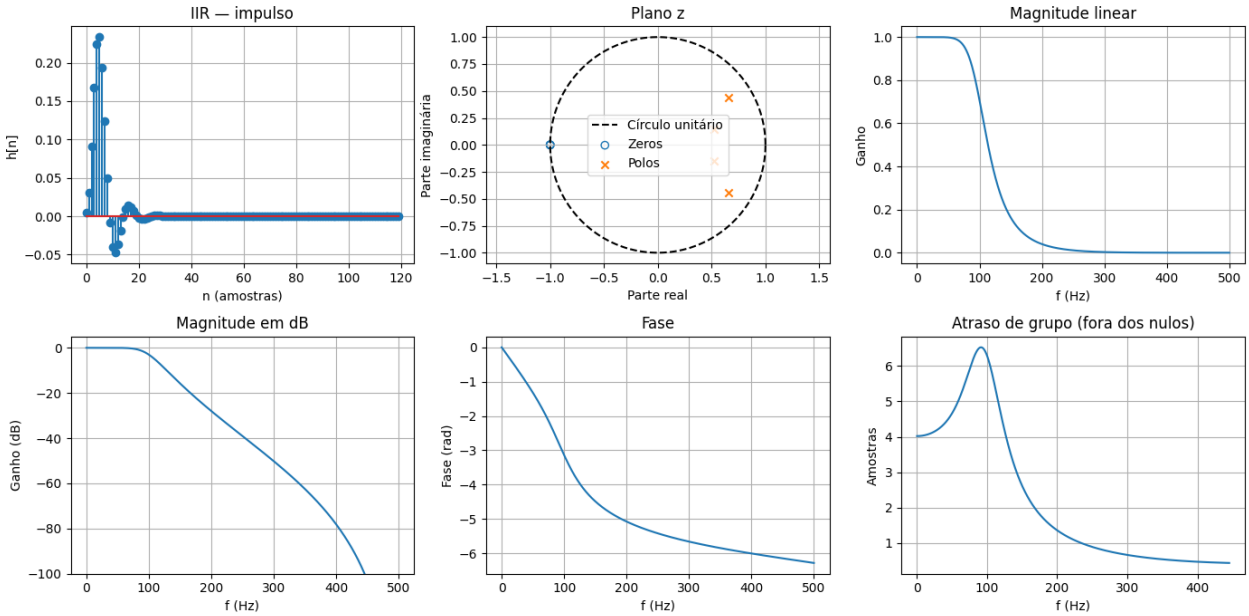
def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

fs=1000; bfir=signal.firwin(51,100,fs=fs); biir,aiir=signal.butter(4,100,fs=fs)
for nome,b,a in [('FIR',bfir,[1]),('IIR',biir,aiir)]:
    analisar(b,a,fs,nome)
    print(nome,'ordem =',max(len(b),len(a)) -1,'coeficientes armazenados =',len(b)+len(a) -1,'multiplicações
     /amostra (forma direta) =',len(b)+len(a) -1)
```




```
b = [ 6.22233219e -19  6.50395537e -04  1.24025458e -03  1.54168857e -03
  1.21680726e -03 -1.30554005e -18 -2.01071740e -03 -4.14071058e -03
 -5.21455172e -03 -4.01474662e -03  3.09446055e -18  6.05036949e -03
  1.18742206e -02  1.43176216e -02  1.06257168e -02 -5.30568790e -18
 -1.52613457e -02 -2.96911186e -02 -3.59490768e -02 -2.72307378e -02
  7.09460841e -18  4.40087761e -02  9.74197266e -02  1.48825236e -01
  1.85978564e -01  1.99527255e -01  1.85978564e -01  1.48825236e -01
  9.74197266e -02  4.40087761e -02  7.09460841e -18 -2.72307378e -02
 -3.59490768e -02 -2.96911186e -02 -1.52613457e -02 -5.30568790e -18
  1.06257168e -02  1.43176216e -02  1.18742206e -02  6.05036949e -03
  3.09446055e -18 -4.01474662e -03 -5.21455172e -03 -4.14071058e -03
 -2.01071740e -03 -1.30554005e -18  1.21680726e -03 -1.54168857e -03
  1.24025458e -03  6.50395537e -04  6.22233219e -19]
a = [1]
Polos: [] Estável causal: True
FIR ordem = 50 coeficientes armazenados = 51 multiplicações/amostra (forma direta) = 51
```




```
b = [0.00482434 0.01929737 0.02894606 0.01929737 0.00482434]
a = [ 1.         -2.36951301  2.31398841 -1.05466541  0.18737949]
Polos: [0.66045672+0.44332349j 0.66045672 -0.44332349j 0.52429979+0.1457741j
 0.52429979 -0.1457741j ] Estável causal: True
IIR ordem = 4 coeficientes armazenados = 9 multiplicações/amostra (forma direta) = 9
```

---

71 Resultados

O FIR possui inclinação de fase correspondente a 25 amostras de atraso; o IIR tem inclinação variável.

72 Interpretação e Discussão

Não se interpreta fase em nulos de magnitude. Em sinais de pulso ou formas de onda com várias componentes, atrasos diferentes por frequência podem modificar o formato temporal.

73 Conclusões

O FIR possui inclinação de fase correspondente a 25 amostras de atraso; o IIR tem inclinação variável.

74 Atraso de Grupo

---

75 Resumo conceitual

O atraso de grupo mede a inclinação negativa da fase e descreve o atraso de envoltórias estreitas em frequência. Sua constância na banda útil é relevante para preservar formas de onda.

---

76 Formulação matemática

$$\tau_g(\omega) = -\frac{d\phi}{d\omega}, \qquad t_g = \tau_g / f_s. \tag{10}$$

A derivada em radianos por amostra resulta em amostras; dividir por fs fornece segundos.

---

77 Exemplo

Compare os atrasos do FIR de 51 coeficientes e do IIR de ordem 4. Observe apenas a banda passante e exclua nulos da interpretação.

---

78 Simulação

Compare os atrasos do FIR de 51 coeficientes e do IIR de ordem 4. Observe apenas a banda passante e exclua nulos da interpretação.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

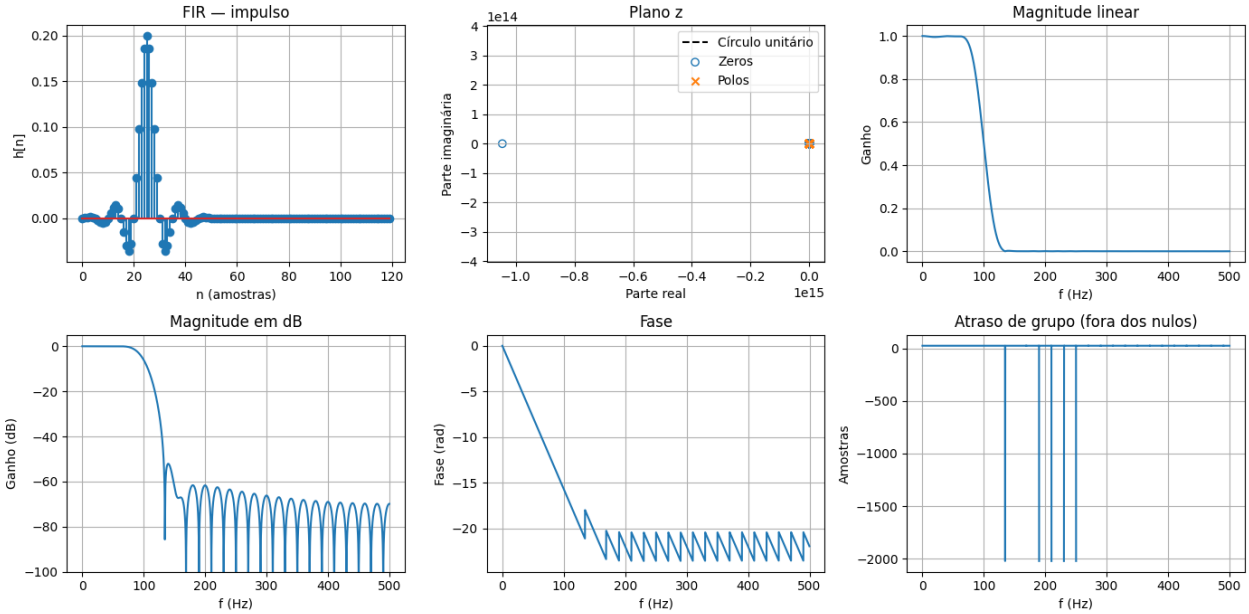
```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

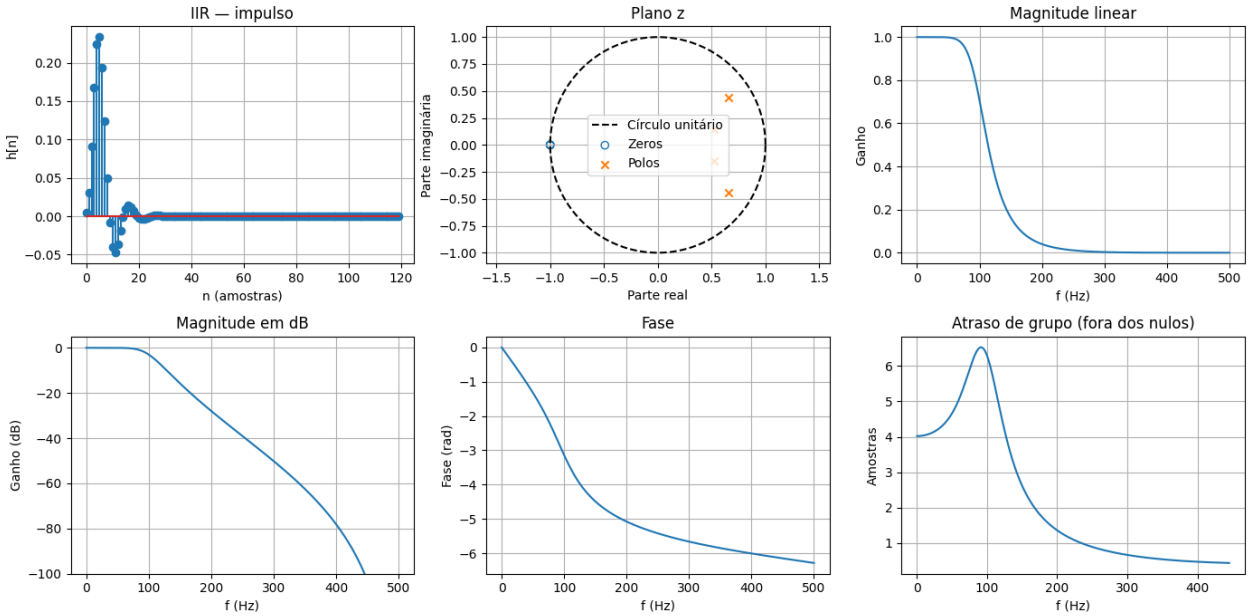
def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

fs=1000; bfir=signal.firwin(51,100,fs=fs); biir,aiir=signal.butter(4,100,fs=fs)
for nome,b,a in [('FIR',bfir,[1]),('IIR',biir,aiir)]:
    analisar(b,a,fs,nome)
    print(nome,'ordem =',max(len(b),len(a)) -1,'coeficientes armazenados =',len(b)+len(a) -1,'multiplicações
     /amostra (forma direta) =',len(b)+len(a) -1)
```




```
b = [ 6.22233219e -19  6.50395537e -04  1.24025458e -03  1.54168857e -03
  1.21680726e -03 -1.30554005e -18 -2.01071740e -03 -4.14071058e -03
 -5.21455172e -03 -4.01474662e -03  3.09446055e -18  6.05036949e -03
  1.18742206e -02  1.43176216e -02  1.06257168e -02 -5.30568790e -18
 -1.52613457e -02 -2.96911186e -02 -3.59490768e -02 -2.72307378e -02
  7.09460841e -18  4.40087761e -02  9.74197266e -02  1.48825236e -01
  1.85978564e -01  1.99527255e -01  1.85978564e -01  1.48825236e -01
  9.74197266e -02  4.40087761e -02  7.09460841e -18 -2.72307378e -02
 -3.59490768e -02 -2.96911186e -02 -1.52613457e -02 -5.30568790e -18
  1.06257168e -02  1.43176216e -02  1.18742206e -02  6.05036949e -03
  3.09446055e -18 -4.01474662e -03 -5.21455172e -03 -4.14071058e -03
 -2.01071740e -03 -1.30554005e -18  1.21680726e -03 -1.54168857e -03
  1.24025458e -03  6.50395537e -04  6.22233219e -19]
a = [1]
Polos: [] Estável causal: True
FIR ordem = 50 coeficientes armazenados = 51 multiplicações/amostra (forma direta) = 51
```




```
b = [0.00482434 0.01929737 0.02894606 0.01929737 0.00482434]
a = [ 1.         -2.36951301  2.31398841 -1.05466541  0.18737949]
Polos: [0.66045672+0.44332349j 0.66045672 -0.44332349j 0.52429979+0.1457741j
 0.52429979 -0.1457741j ] Estável causal: True
IIR ordem = 4 coeficientes armazenados = 9 multiplicações/amostra (forma direta) = 9
```

79 Resultados

O FIR tem atraso de 25 amostras, equivalente a 25 ms a 1000 Hz. O atraso do IIR varia com a frequência.

80 Interpretação e Discussão

O atraso de grupo não deve ser usado em regiões onde a magnitude é praticamente nula. Filtros de baixa latência podem justificar fase não linear, enquanto aquisição de pulsos pode exigir maior uniformidade de atraso.

81 Conclusões

O FIR tem atraso de 25 amostras, equivalente a 25 ms a 1000 Hz. O atraso do IIR varia com a frequência.

82 Projeto de Filtros FIR

---

83 Resumo conceitual

O método de janelas aproxima uma resposta ideal infinita por coeficientes finitos. A janela controla os lóbulos laterais e o comprimento controla a largura da transição.

---

84 Formulação matemática

$$h_d[n] = 2\frac{f_c}{f_s}\,\mathrm{sinc}\left(2\frac{f_c}{f_s}(n - M/2)\right), \quad h[n] = h_d[n]w[n]. \tag{11}$$

Aqui sinc usa $\sin(\pi x)/(\pi x)$; a ordem é M e existem M+1 coeficientes.

---

85 Exemplo

Projete passa-baixas de 100 Hz a fs = 1000 Hz, compare ordens 30 e 60 e janelas retangular, Hann, Hamming e Blackman.

---

86 Simulação

Projete passa-baixas de 100 Hz a fs = 1000 Hz, compare ordens 30 e 60 e janelas retangular, Hann e Hamming.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

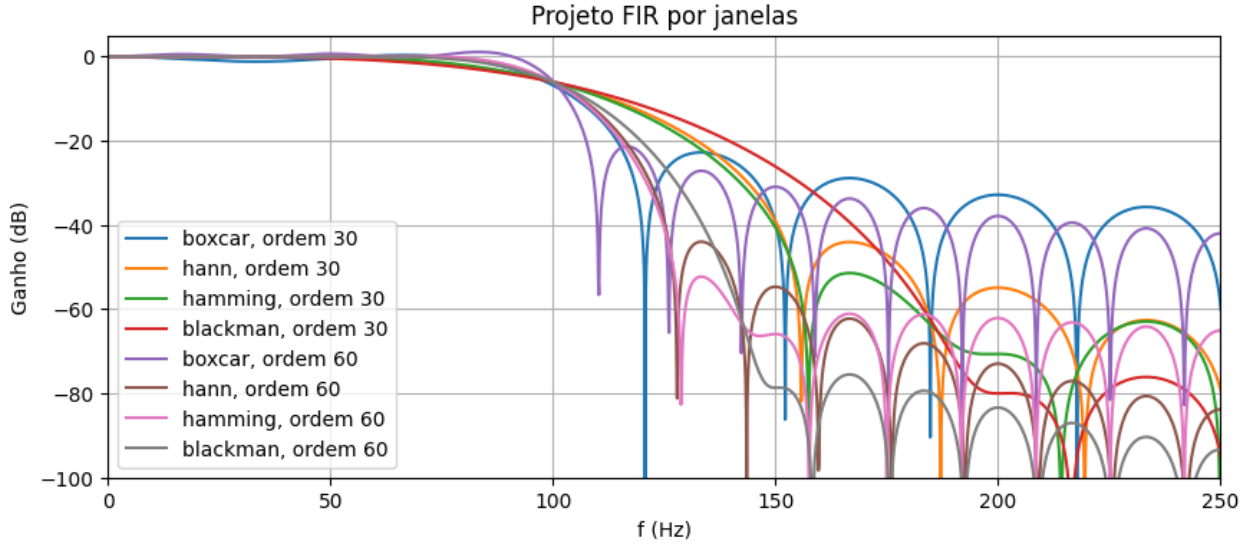
def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

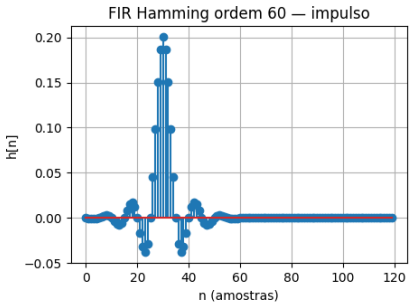
def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

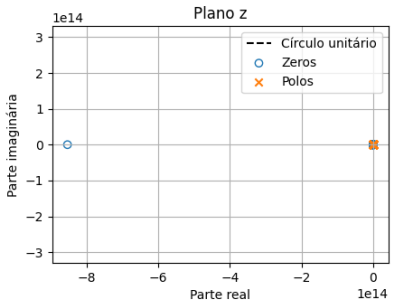
for L in [31,61]:
    for janela in ['boxcar','hann','hamming','blackman']:
        b=signal.firwin(L,100,window=janela,fs=1000)
        f,H=signal.freqz(b,fs=1000,worN=4096)
        plt.plot(f,20*np.log10(np.maximum(abs(H),1e -8)),label=f'{janela}, ordem {L -1}')
        print(janela,'ordem',L -1,'soma dos coeficientes',b.sum(),'simetria:',np.allclose(b,b[:: -1]))
plt.xlim(0,250); plt.ylim( -100,5); plt.xlabel('f (Hz)'); plt.ylabel('Ganho (dB)'); plt.title('Projeto FIR
     por janelas'); plt.legend(); plt.show()
analisar(signal.firwin(61,100,fs=1000),fs=1000,nome='FIR Hamming ordem 60')
```

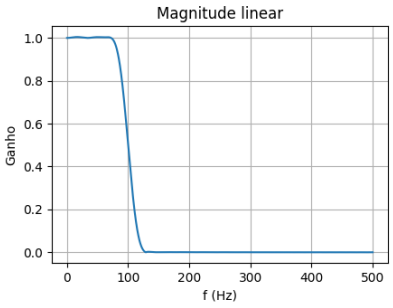
```
boxcar ordem 30 soma dos coeficientes 1.0000000000000002 simetria: True
hann ordem 30 soma dos coeficientes 1.0000000000000002 simetria: True
hamming ordem 30 soma dos coeficientes 1.0 simetria: True
blackman ordem 30 soma dos coeficientes 0.9999999999999998 simetria: True
boxcar ordem 60 soma dos coeficientes 0.9999999999999999 simetria: True
hann ordem 60 soma dos coeficientes 1.0 simetria: True
hamming ordem 60 soma dos coeficientes 0.9999999999999999 simetria: True
```

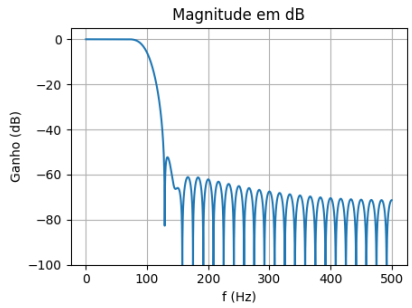
```
blackman ordem 60 soma dos coeficientes 1.0 simetria: True
```

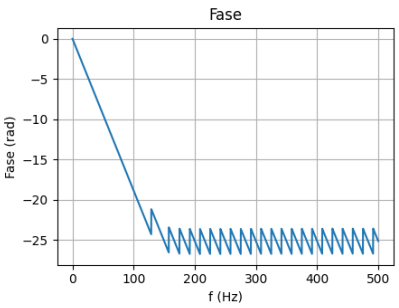


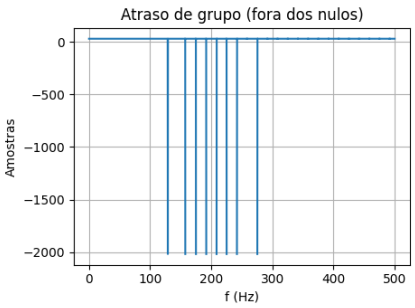













```
b = [ -6.25062962e -19 -5.33546726e -04 -9.75742199e -04 -1.15191036e -03
 -8.63739968e -04  1.10658267e -18  1.31137641e -03  2.61380332e -03
  3.20212767e -03  2.40736791e -03 -2.42211898e -18 -3.48266003e -03
 -6.70576757e -03 -7.93023216e -03 -5.76478856e -03  4.21917499e -18
  7.87630045e -03  1.48342877e -02  1.72463226e -02  1.23940257e -02
 -6.01623101e -18 -1.68833441e -02 -3.21516098e -02 -3.82203158e -02
 -2.85052982e -02  7.33176731e -18  4.50118960e -02  9.88527024e -02
  1.50169453e -01  1.87031968e -01  2.00434649e -01  1.87031968e -01
  1.50169453e -01  9.88527024e -02  4.50118960e -02  7.33176731e -18
 -2.85052982e -02 -3.82203158e -02 -3.21516098e -02 -1.68833441e -02
 -6.01623101e -18  1.23940257e -02  1.72463226e -02  1.48342877e -02
  7.87630045e -03  4.21917499e -18 -5.76478856e -03 -7.93023216e -03
 -6.70576757e -03 -3.48266003e -03 -2.42211898e -18  2.40736791e -03
  3.20212767e -03  2.61380332e -03  1.31137641e -03  1.10658267e -18
 -8.63739968e -04 -1.15191036e -03 -9.75742199e -04 -5.33546726e -04
 -6.25062962e -19]
a = [1]
Polos: [] Estável causal: True
```

87 Resultados

O comprimento maior estreita a transição. Janelas suaves reduzem lóbulos laterais em relação à retangular; os coeficientes são simétricos e somam aproximadamente 1.

88 Interpretação e Discussão

A frequência fornecida a firwin indica meia amplitude aproximadamente, não meia potência. A validação deve medir a banda passante e a rejeição em uma grade densa, em vez de assumir atendimento com base apenas na janela.

89 Conclusões

O comprimento maior estreita a transição. Janelas suaves reduzem lóbulos laterais em relação à retangular; os coeficientes são simétricos e somam aproximadamente 1.

90 Projeto de Filtros IIR

---

91 Resumo conceitual

O projeto IIR aproxima especificações com famílias de funções racionais. Butterworth privilegia magnitude monotônica; Chebyshev I permite ripple na passagem; Chebyshev II permite ripple na rejeição; elíptico permite ripple nas duas bandas para obter transições mais seletivas.

---

92 Formulação matemática

$$H(z) = \frac{\sum_{k=0}^{M} b_k z^{-k}}{1 + \sum_{k=1}^{N} a_k z^{-k}}. \tag{12}$$

Especificações: banda passante até f_p, rejeição a partir de f_r, perda máxima A_p e atenuação mínima A_r.

---

93 Exemplo

Compare Butterworth, Chebyshev I, Chebyshev II e elíptico para fs = 1000 Hz, fp = 80 Hz, fr = 150 Hz, Ap = 1 dB e Ar = 40 dB.

---

94 Simulação

Compare Butterworth, Chebyshev I e elíptico para fs = 1000 Hz, fp = 80 Hz, fr = 150 Hz, Ap = 1 dB e Ar = 40 dB.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

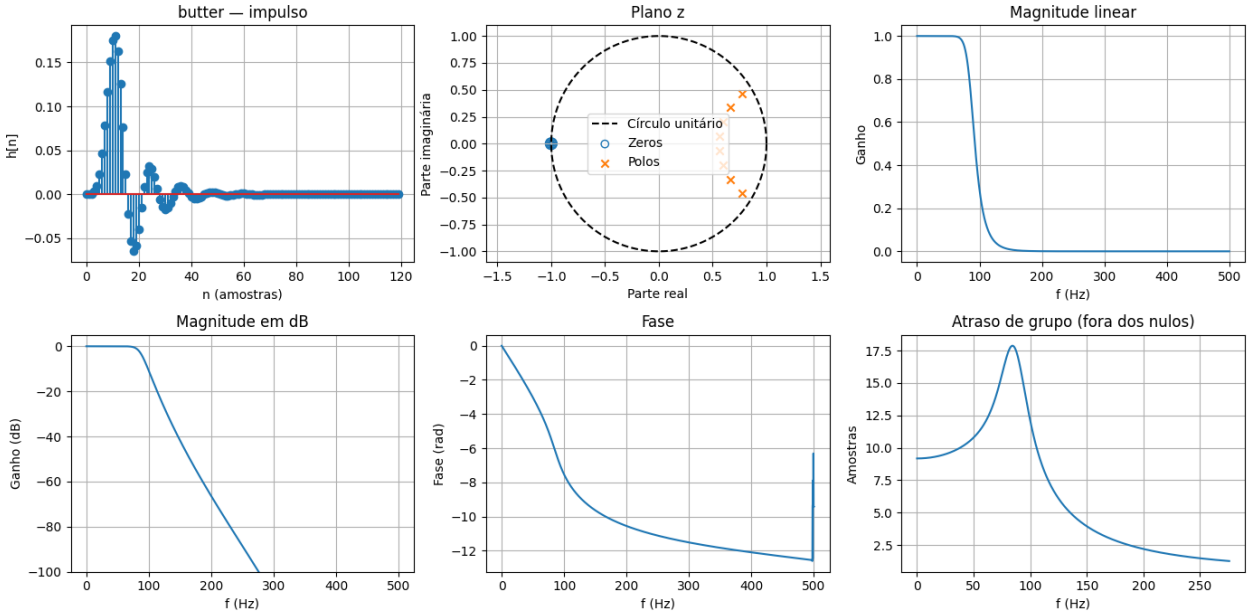
```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

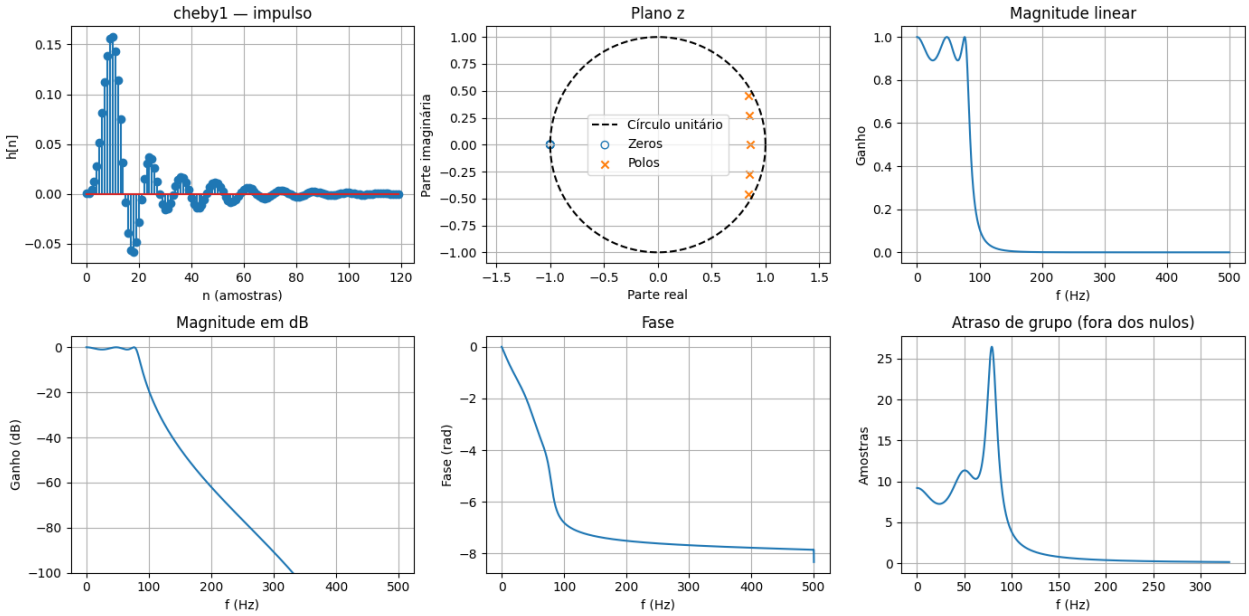
def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

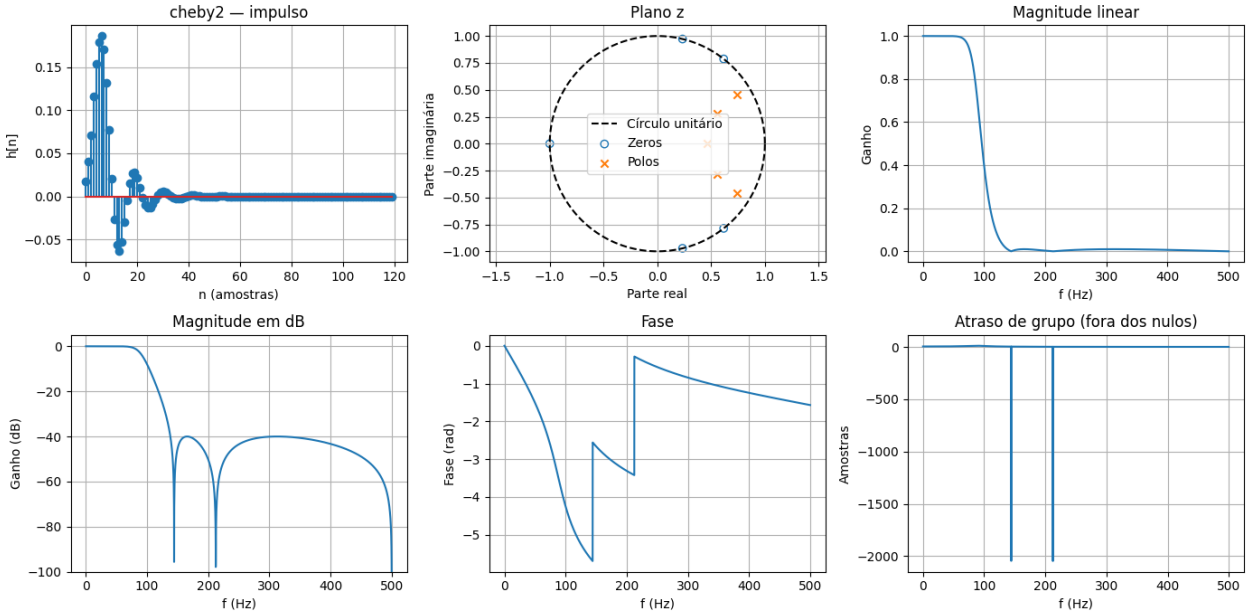
for familia in ['butter','cheby1','cheby2','ellip']:
    b,a=signal.iirdesign(80,150,1,40,fs=1000,ftype=familia)
    analisar(b,a,1000,familia)
    print(familia,'ordem:',len(a) -1)
```



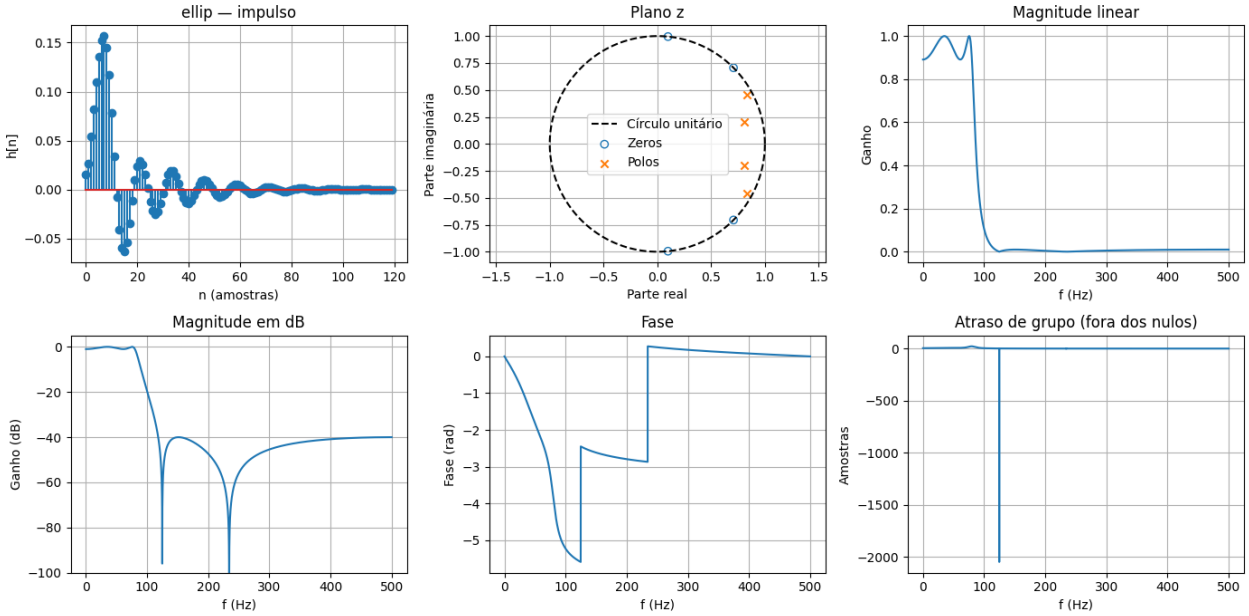

```
b = [8.97754728e -06 7.18203783e -05 2.51371324e -04 5.02742648e -04
 6.28428310e -04 5.02742648e -04 2.51371324e -04 7.18203783e -05
 8.97754728e -06]
a = [  1.          -5.21053433  12.22219774 -16.76747247  14.66317686
  -8.34855367   3.01616106  -0.63117624   0.05849931]
Polos: [0.77666109+0.46166593j 0.77666109 -0.46166593j 0.66399355+0.33460512j
 0.66399355 -0.33460512j 0.59763829+0.20123324j 0.59763829 -0.20123324j
 0.56697424+0.06703805j 0.56697424 -0.06703805j] Estável causal: True
butter ordem: 8
```




```
b = [9.99665501e -05 4.99832751e -04 9.99665501e -04 9.99665501e -04
 4.99832751e -04 9.99665501e -05]
a = [ 1.         -4.23639803  7.46293092 -6.80728271  3.20880553 -0.62485679]
Polos: [0.84137669+0.45759741j 0.84137669 -0.45759741j 0.84600899+0.2735829j
 0.84600899 -0.2735829j  0.86162667+0.j        ] Estável causal: True
cheby1 ordem: 5
```




```
b = [ 0.01732778 -0.01217906  0.01517478  0.01517478 -0.01217906  0.01732778]
a = [ 1.         -3.0701349   4.03014223 -2.74781976  0.96773393 -0.1392745 ]
Polos: [0.74407367+0.45995666j 0.74407367 -0.45995666j 0.55820066+0.28166796j
 0.55820066 -0.28166796j 0.46558623+0.j        ] Estável causal: True
cheby2 ordem: 5
```




```
b = [ 0.01570675 -0.02536928  0.03583375 -0.02536928  0.01570675]
a = [ 1.         -3.28057711  4.28424057 -2.60945179  0.62431139]
Polos: [0.83416324+0.45565421j 0.83416324 -0.45565421j 0.80612532+0.20296074j
 0.80612532 -0.20296074j] Estável causal: True
ellip ordem: 4
```

---

95 Resultados

As quatro famílias produzem respostas distintas para o mesmo conjunto de tolerâncias. A família elíptica tende a exigir menos coeficientes, com ripple e fase não linear.

96 Interpretação e Discussão

A função de projeto calcula coeficientes para as especificações; cabe verificar bandas, polos e efeito temporal. Para ordens elevadas, seções de segunda ordem reduzem problemas numéricos da forma polinomial direta.

97 Conclusões

As quatro famílias produzem respostas distintas para o mesmo conjunto de tolerâncias. A família elíptica tende a exigir menos coeficientes, com ripple e fase não linear.

98 Comparação de Métodos de Projeto

---

99 Resumo conceitual

A comparação de métodos deve fixar as mesmas bandas e tolerâncias. Ordem menor, fase linear, atraso e robustez numérica são critérios complementares.

---

100 Formulação matemática

$$A_r = -\max_{f \geq f_r} 20 \log_{10} |H(f)|, \quad R_p = \max_{f \leq f_p} G_{dB} - \min_{f \leq f_p} G_{dB}. \tag{13}$$

A verificação usa grade de 32768 pontos; os extremos entre pontos podem diferir ligeiramente.

---

101 Exemplo

Use fp = 80 Hz, fr = 150 Hz, fs = 1000 Hz, Ap = 1 dB e Ar = 40 dB; procure um comprimento FIR Hamming suficiente e compare quatro famílias IIR.

---

102 Simulação

Use fp = 80 Hz, fr = 150 Hz, fs = 1000 Hz, Ap = 1 dB e Ar = 40 dB; procure um comprimento FIR Hamming suficiente e compare três famílias IIR.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
```

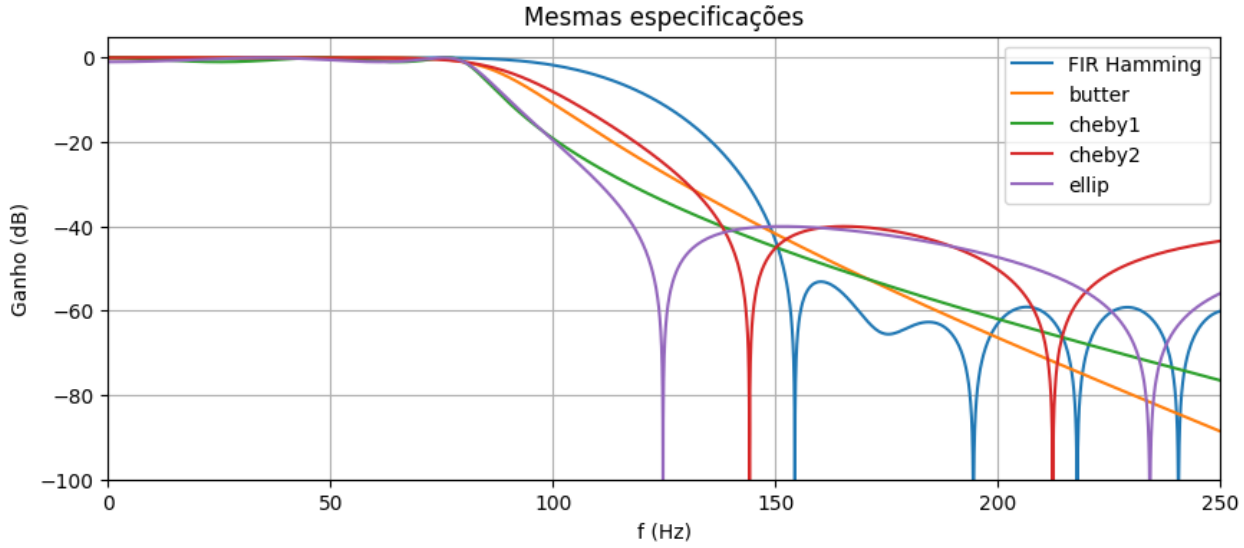
```
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

fs=1000; linhas=[]
for L in range(11,402,2):
    b=signal.firwin(L,115,fs=fs); f,H=signal.freqz(b,worN=32768,fs=fs); db=20*np.log10(np.maximum(abs(H),1e
     -15))
    if db[f<=80].min()>= -1 and db[f<=80].max()<=1 and db[f>=150].max()<= -40: break
filtros=[('FIR Hamming',b,np.array([1.]))]
for familia in ['butter','cheby1','cheby2','ellip']:
    bi,ai=signal.iirdesign(80,150,1,40,fs=fs,ftype=familia); filtros.append((familia,bi,ai))
for nome,b,a in filtros:
    f,H=signal.freqz(b,a,worN=32768,fs=fs); db=20*np.log10(np.maximum(abs(H),1e -15)); pb=db[f<=80]; ar= -db
     [f>=150].max()
    linhas.append([nome,max(len(b),len(a)) -1,len(b)+len(a) -1,pb.max() -pb.min(),ar])
    plt.plot(f,db,label=nome)
display(pd.DataFrame(linhas,columns=['Método','Ordem','Mult./am.','Ripple (dB)','Atenuação (dB)']))
plt.xlim(0,250); plt.ylim( -100,5); plt.xlabel('f (Hz)'); plt.ylabel('Ganho (dB)'); plt.title('Mesmas
     especificações'); plt.legend(); plt.show()
```

	Método	Ordem	Mult./am.	Ripple (dB)	Atenuação (dB)
0	FIR Hamming	44	45	0.077623	43.326879
1	butter	8	17	0.997500	41.760124
2	cheby1	5	11	1.000000	44.918219
3	cheby2	5	11	0.998121	40.000000
4	ellip	4	9	1.000000	40.000000



---

103 Resultados

A tabela calcula ordem, custo direto, ripple e atenuação obtidos. O FIR é selecionado por verificação explícita das bandas; os IIR são calculados para as mesmas tolerâncias.

104 Interpretação e Discussão

Para um sensor com prioridade de baixo custo, um IIR é atraente. Se a preservação temporal for decisiva, a fase linear do FIR pode justificar mais coeficientes e atraso. A escolha final requer analisar também os sinais processados.

105 Conclusões

A tabela calcula ordem, custo direto, ripple e atenuação obtidos. O FIR é selecionado por verificação explícita das bandas; os IIR são calculados para as mesmas tolerâncias.

106 Aplicação de Filtros a Sinais

---

107 Resumo conceitual

A análise espectral deve preceder o projeto: identificar sinal útil e interferência permite justificar o tipo e as bandas do filtro. A aplicação causal também introduz transientes e atraso.

---

108 Formulação matemática

$$x[n] = \cos(2\pi 40 n/f_s) + 0{,}6\cos(2\pi 200 n/f_s) + r[n], \quad Y = XH. \tag{14}$$

O objetivo é preservar 40 Hz e reduzir 200 Hz, com ruído gaussiano de desvio padrão 0,1.

---

109 Exemplo

Gere 4 s a 1000 Hz com semente 42 e aplique um FIR Hamming de 61 coeficientes com parâmetro de corte de 115 Hz.

---

110 Simulação

Gere 4 s a 1000 Hz com semente 42 e aplique um FIR Hamming de 61 coeficientes com parâmetro de corte de 115 Hz.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

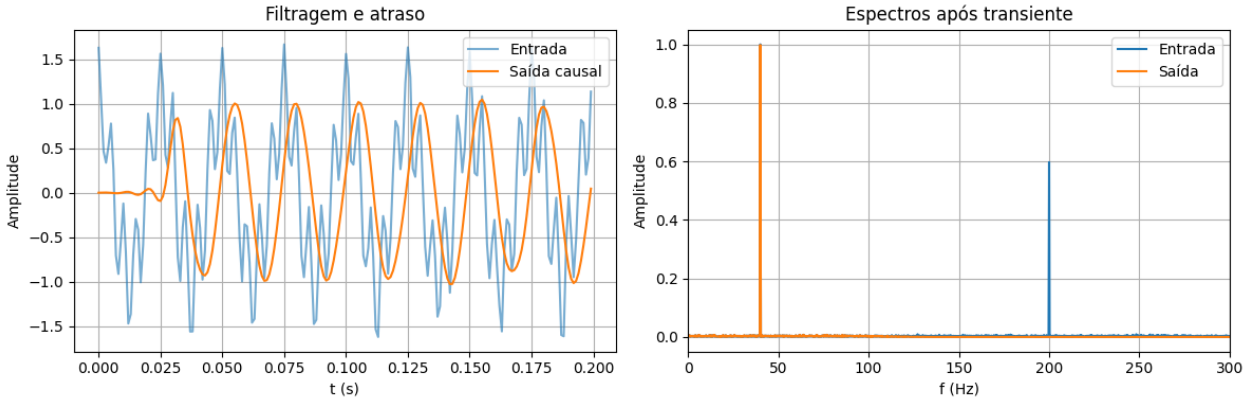
def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
```

```
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

fs=1000; t=np.arange(4000)/fs; ref=np.cos(2*np.pi*40*t)
x=ref+.6*np.cos(2*np.pi*200*t)+np.random.default_rng(42).normal(0,.1,len(t))
b=signal.firwin(61,115,fs=fs); y=signal.lfilter(b,[1],x)
fig,ax=plt.subplots(1,2,figsize=(12,4))
ax[0].plot(t[:200],x[:200],alpha=.6,label='Entrada'); ax[0].plot(t[:200],y[:200],label='Saída causal')
for nome,v in [('Entrada',x),('Saída',y)]:
    f,A=espectro(v[500:],fs); ax[1].plot(f,A,label=nome)
ax[0].set(xlabel='t (s)',ylabel='Amplitude',title='Filtragem e atraso'); ax[1].set(xlabel='f (Hz)',ylabel='
     Amplitude',title='Espectros após transiente',xlim=(0,300))
for a in ax: a.legend()
plt.tight_layout(); plt.show()
for fi in [40,200]:
    _,H=signal.freqz(b,worN=np.array([2*np.pi*fi/fs])); print(fi,'Hz: ganho =',abs(H[0]))
```




```
40 Hz: ganho = 0.9984243587152845
200 Hz: ganho = 0.000623247119368365
```

---

111 Resultados

O espectro mostra preservação de 40 Hz e forte redução de 200 Hz. O sinal temporal filtrado aparece deslocado por 30 amostras.

112 Interpretação e Discussão

O atraso não deve ser confundido com erro de amplitude. Para medir erro em relação à referência, é necessário alinhar os sinais e excluir o transiente; o mini projeto apresenta esse procedimento para FIR e IIR.

113 Conclusões

O espectro mostra preservação de 40 Hz e forte redução de 200 Hz. O sinal temporal filtrado aparece deslocado por 30 amostras.

114 Mini Projeto Integrador — Tratamento Digital de um Sensor

---

115 Resumo conceitual

O problema é reduzir uma interferência de 200 Hz em um sensor preservando a componente útil de 40 Hz. A origem dos dados é um modelo matemático com ruído pseudoaleatório reproduzível, sem uso de medições reais.

---

116 Formulação matemática

$$x_{ref}[n] = \cos(2\pi 40 n/1000), \quad x[n] = x_{ref}[n] + 0{,}6\cos(2\pi 200 n/1000) + r[n]. \tag{15}$$

Define-se $f_p = 80$ Hz, $f_r = 150$ Hz, perda máxima de 1 dB e rejeição mínima de 40 dB. As métricas são $MSE = N^{-1}\sum_n (y[n] - x_{ref}[n])^2$, $SNR = 10\log_{10}(\sum x_{ref}^2 / \sum (y - x_{ref})^2)$ e ganho na componente de interesse.

---

117 Exemplo

O sinal dura 4 s a fs = 1000 Hz (Δf = 0,25 Hz), com ruído gaussiano de desvio 0,1 e semente 42. A banda passante até 80 Hz inclui 40 Hz com margem. A rejeição começa antes de 200 Hz, permitindo transição de 70 Hz. Compare FIR Hamming de comprimento ímpar escolhido por validação com IIR Butterworth de ordem calculada.

---

118 Simulação

O sinal dura 4 s a fs = 1000 Hz (Δf = 0,25 Hz), com ruído gaussiano de desvio 0,1 e semente 42. A banda passante até 80 Hz inclui 40 Hz com margem. A rejeição começa antes de 200 Hz, permitindo transição de 70 Hz. Compare FIR Hamming de comprimento ímpar escolhido por validação com IIR Butterworth de ordem calculada.
Execute as células em ordem. Os dados são sintéticos; não representam medições experimentais.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import pandas as pd
from IPython.display import display
plt.rcParams.update({'figure.figsize': (10, 4), 'axes.grid': True})

def espectro(x, fs, janela=None, nfft=None):
    w = np.ones(len(x)) if janela is None else janela
    X = np.fft.rfft(x*w, n=nfft)
    f = np.fft.rfftfreq(len(x) if nfft is None else nfft, 1/fs)
    A = np.abs(X)/w.sum()
    A[1:] *= 2
    if (len(x) if nfft is None else nfft) % 2 == 0:
        A[ -1] /= 2
    return f, A

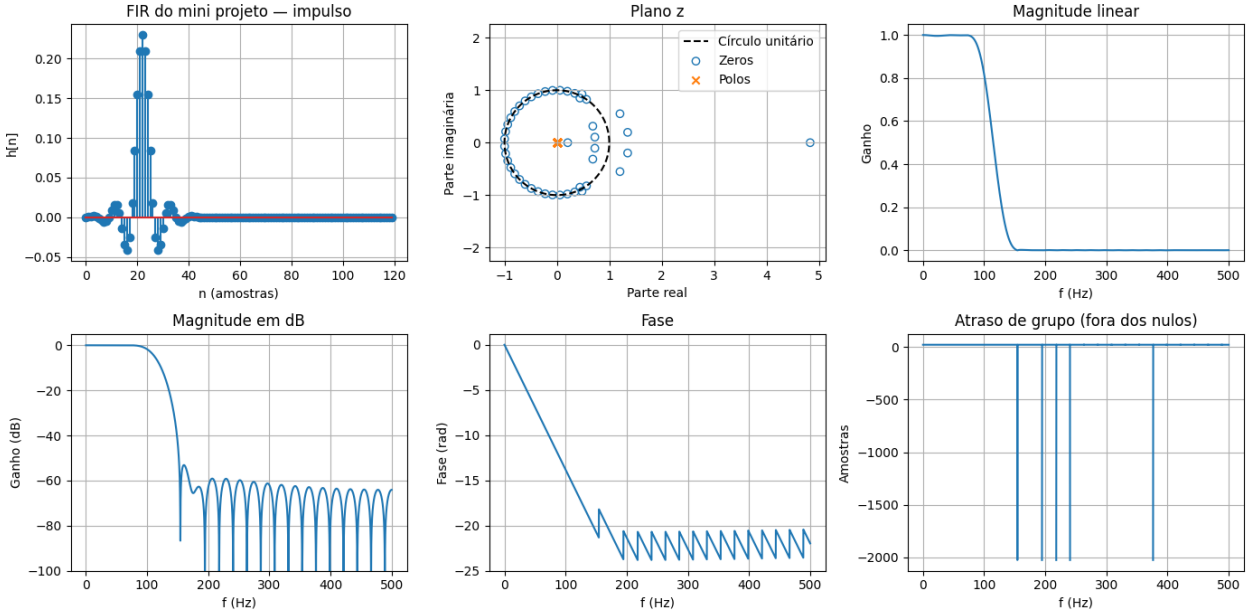
def plano(b, a, ax):
    # Coeficientes em potências de z^ -1; completar graus para exibir polos na origem.
    grau = max(len(b), len(a)) -1
    z = np.roots(np.pad(b, (0, grau+1 -len(b))))
    p = np.roots(np.pad(a, (0, grau+1 -len(a))))
    theta = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(theta), np.sin(theta), 'k - -', label='Círculo unitário')
```

```
    ax.scatter(z.real, z.imag, facecolors='none', edgecolors='C0', label='Zeros')
    ax.scatter(p.real, p.imag, marker='x', color='C1', label='Polos')
    ax.set(xlabel='Parte real', ylabel='Parte imaginária', title='Plano z')
    ax.axis('equal'); ax.legend()

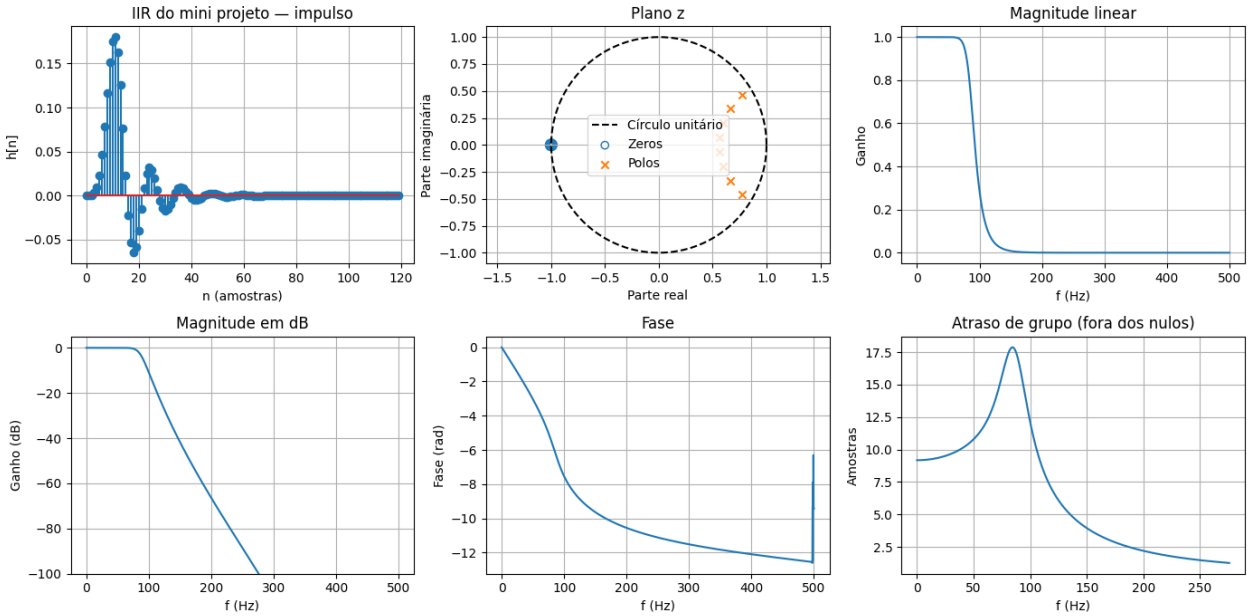
def analisar(b, a=(1,), fs=1000, nome='Filtro'):
    w, H = signal.freqz(b, a, worN=4096, fs=fs)
    impulso = np.r_[1., np.zeros(119)]
    h = signal.lfilter(b, a, impulso)
    fig, ax = plt.subplots(2, 3, figsize=(14, 7))
    ax[0,0].stem(np.arange(len(h)), h); ax[0,0].set(title=nome+' — impulso', xlabel='n (amostras)', ylabel='
     h[n]')
    plano(np.asarray(b), np.asarray(a), ax[0,1])
    ax[0,2].plot(w, np.abs(H)); ax[0,2].set(title='Magnitude linear', xlabel='f (Hz)', ylabel='Ganho')
    ax[1,0].plot(w, 20*np.log10(np.maximum(np.abs(H), 1e -12))); ax[1,0].set(title='Magnitude em dB', xlabel
     ='f (Hz)', ylabel='Ganho (dB)', ylim=( -100,5))
    fase = np.unwrap(np.angle(H))
    ax[1,1].plot(w, fase); ax[1,1].set(title='Fase', xlabel='f (Hz)', ylabel='Fase (rad)')
    gd = -np.gradient(fase, 2*np.pi*w/fs)
    gd[np.abs(H)<1e -5] = np.nan
    ax[1,2].plot(w, gd); ax[1,2].set(title='Atraso de grupo (fora dos nulos)', xlabel='f (Hz)', ylabel='
     Amostras')
    fig.tight_layout(); plt.show()
    print('b =', np.asarray(b)); print('a =', np.asarray(a))
    print('Polos:', np.roots(a), 'Estável causal:', np.all(np.abs(np.roots(a)) < 1))

fs=1000; t=np.arange(4000)/fs; ref=np.cos(2*np.pi*40*t)
x=ref+.6*np.cos(2*np.pi*200*t)+np.random.default_rng(42).normal(0,.1,len(t))
for L in range(11,402,2):
    bf=signal.firwin(L,115,window='hamming',fs=fs)
    f,H=signal.freqz(bf,worN=32768,fs=fs); db=20*np.log10(np.maximum(abs(H),1e -15))
    if db[f<=80].min()>= -1 and db[f<=80].max()<=1 and db[f>=150].max()<= -40: break
else: raise RuntimeError('FIR não atende às especificações')
N,Wn=signal.buttord(80,150,1,40,fs=fs)
bi,ai=signal.butter(N,Wn,fs=fs)
sos=signal.butter(N,Wn,fs=fs,output='sos')
yf=signal.lfilter(bf,[1],x); yi=signal.sosfilt(sos,x)
# Validação espectral nas mesmas bandas, antes de medir erros no tempo.
linhas=[]
for nome,b,a,y in [('FIR',bf,np.array([1.]),yf),('IIR',bi,ai,yi)]:
    f,H=signal.freqz(b,a,worN=32768,fs=fs); db=20*np.log10(np.maximum(abs(H),1e -15)); pb=db[f<=80]; ar= -db
     [f>=150].max()
    assert pb.min()>= -1.001 and pb.max()<=1.001 and ar>=39.999
    assert np.all(np.abs(np.roots(a))<1)
    # Alinhamento por fase na frequência útil; não equivale a retirar a distorção de fase de banda larga.
    ww,HH=signal.freqz(b,a,worN=np.array([2*np.pi*40/fs,2*np.pi*200/fs]))
    atraso= int(round( -np.angle(HH[0])/(2*np.pi*40/fs))) if nome=='IIR' else (len(b) -1)//2
    inicio=500; fim=len(x) -100 -max(atraso,0)
    alvo=ref[inicio:fim]; medido=y[inicio+atraso:fim+atraso]
    mse=np.mean((medido -alvo)**2); snr=10*np.log10(np.mean(alvo**2)/mse)
    mse_bruto=np.mean((y[inicio:fim] -alvo)**2)
    w,Ha=signal.freqz(b,a,worN=8192,fs=fs); gd= -np.gradient(np.unwrap(np.angle(Ha)),2*np.pi*w/fs)
    group=gd[w<=80]
    linhas.append([nome,max(len(b),len(a)) -1,len(b)+len(a) -1,len(b)+len(a) -1,len(b)+len(a) -2,ar,pb.max()
      -pb.min(),group.min(),group.max(),atraso,abs(HH[0]),20*np.log10(abs(HH[1])),mse_bruto,mse,snr])
    analisar(b,a,fs,nome+' do mini projeto')
    print(nome,'equação: y[n] = soma b[k]x[n -k] - soma a[k]y[n -k], a[0]=1')
df=pd.DataFrame(linhas,columns=['Filtro','Ordem','Coeficientes (sem a0)','Multiplicações/amostra','Somas/
     amostra','Atenuação rejeição (dB)','Ripple (dB)','Atraso grupo mín. (amostras)','Atraso grupo máx. (
     amostras)','Alinhamento (amostras)','Ganho 40 Hz','Ganho 200 Hz (dB)','MSE sem alinhar','MSE alinhado','
     SNR alinhada (dB)'])
display(df.T)
inicio=500; fim=3800; mse0=np.mean((x[inicio:fim] -ref[inicio:fim])**2)
print('Entrada: MSE =',mse0,'SNR =',10*np.log10(np.mean(ref[inicio:fim]**2)/mse0),'dB')
fig,ax=plt.subplots(2,2,figsize=(13,8))
for nome,v in [('Entrada',x),('FIR',yf),('IIR',yi)]:
    ax[0,0].plot(t[500:650],v[500:650],label=nome)
    f,A=espectro(v[500:3500],fs); ax[0,1].plot(f,20*np.log10(np.maximum(A,1e -9)),label=nome)
ax[0,0].plot(t[500:650],ref[500:650],'k - -',label='Referência')
ax[0,0].set(title='Saídas causais, sem alinhamento',xlabel='t (s)',ylabel='Amplitude')
ax[0,1].set(title='Espectros após transiente',xlabel='f (Hz)',ylabel='Amplitude (dB)',xlim=(0,300),ylim=(
     -100,5))
ax[1,0].plot(t[:180],yf[:180],label='FIR'); ax[1,0].plot(t[:180],yi[:180],label='IIR'); ax[1,0].set(title='
     Transiente de partida',xlabel='t (s)',ylabel='Amplitude')
for nome,y,atraso in [('FIR',yf,int(df.iloc[0]['Alinhamento (amostras)'])),('IIR',yi,int(df.iloc[1]['
```

```
    Alinhamento (amostras)'])):
    ax[1,1].plot(t[500:650],y[500+atraso:650+atraso],label=nome+' alinhado')
ax[1,1].plot(t[500:650],ref[500:650],'k - -',label='Referência'); ax[1,1].set(title='Comparação para cálculo
     de erro',xlabel='t de referência (s)',ylabel='Amplitude')
for a in ax.flat: a.legend()
plt.tight_layout(); plt.show()
```




```
b = [ -2.16441858e -04  6.52038586e -04  1.48986617e -03  1.86528200e -03
  1.14978721e -03 -1.00275082e -03 -4.00228301e -03 -6.09290780e -03
 -5.04615194e -03  3.14987484e -04  8.59880634e -03  1.55244791e -02
  1.55594495e -02  5.15537667e -03 -1.39847303e -02 -3.36738817e -02
 -4.14088139e -02 -2.56013956e -02  1.83068904e -02  8.39334270e -02
  1.54636105e -01  2.09081583e -01  2.29522557e -01  2.09081583e -01
  1.54636105e -01  8.39334270e -02  1.83068904e -02 -2.56013956e -02
 -4.14088139e -02 -3.36738817e -02 -1.39847303e -02  5.15537667e -03
  1.55594495e -02  1.55244791e -02  8.59880634e -03  3.14987484e -04
 -5.04615194e -03 -6.09290780e -03 -4.00228301e -03 -1.00275082e -03
  1.14978721e -03  1.86528200e -03  1.48986617e -03  6.52038586e -04
 -2.16441858e -04]
a = [1.]
Polos: [] Estável causal: True
FIR equação: y[n] = soma b[k]x[n -k] - soma a[k]y[n -k], a[0]=1
```

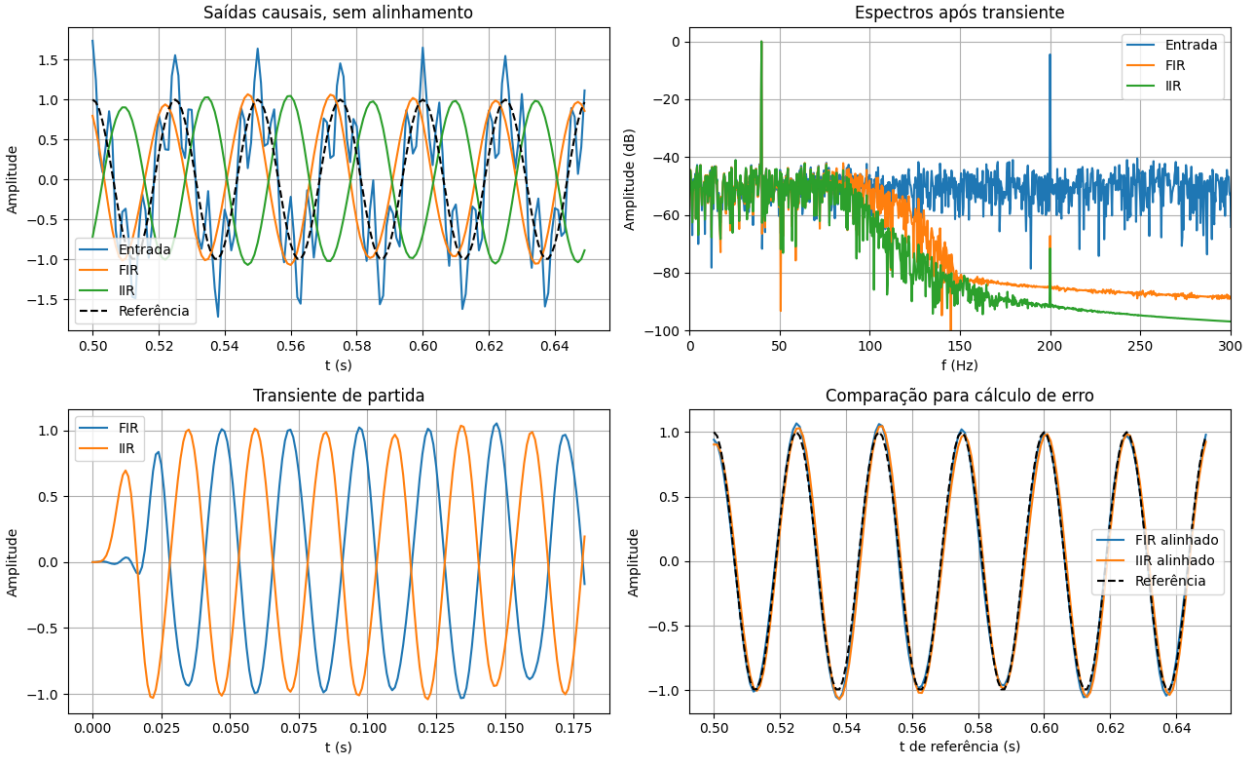



```
b = [8.97754728e -06 7.18203783e -05 2.51371324e -04 5.02742648e -04
 6.28428310e -04 5.02742648e -04 2.51371324e -04 7.18203783e -05
```

```
 8.97754728e -06]
a = [  1.          -5.21053433  12.22219774 -16.76747247  14.66317686
  -8.34855367   3.01616106  -0.63117624   0.05849931]
Polos: [0.77666109+0.46166593j 0.77666109 -0.46166593j 0.66399355+0.33460512j
 0.66399355 -0.33460512j 0.59763829+0.20123324j 0.59763829 -0.20123324j
 0.56697424+0.06703805j 0.56697424 -0.06703805j] Estável causal: True
IIR equação: y[n] = soma b[k]x[n -k] - soma a[k]y[n -k], a[0]=1
```

	0	1
Filtro	FIR	IIR
Ordem	44	8
Coeficientes (sem a0)	45	17
Multiplicações/amostra	45	17
Somas/amostra	44	16
Atenuação rejeição (dB)	43.326879	41.760124
Ripple (dB)	0.077623	0.9975
Atraso grupo mín. (amostras)	22.0	9.173531
Atraso grupo máx. (amostras)	22.0	17.23056
Alinhamento (amostras)	22	9
Ganho 40 Hz	0.999062	0.999998
Ganho 200 Hz (dB)	-63.001089	-66.410121
MSE sem alinhar	0.272318	1.724775
MSE alinhado	0.002253	0.00875
SNR alinhada (dB)	23.465942	17.572654

```
Entrada: MSE = 0.18871318922388677 SNR = 4.23167750133018 dB
```



119 Resultados

A tabela gerada registra as ordens efetivas, coeficientes, custo direto, atenuação, ripple, limites de atraso de grupo, ganho em 40 Hz, ganho em 200 Hz, MSE e SNR. As duas soluções são verificadas por assertions para a mesma especificação. Os gráficos mostram impulso, polos e zeros, magnitude, fase, atraso e sinais antes e depois da filtragem.
Os valores abaixo foram obtidos na execução com semente 42. Os custos referem-se à forma direta geral, sem explorar simetria; a execução IIR usa quatro seções de segunda ordem, com aproximadamente 20 multiplicações e 16 somas por amostra e 20 coeficientes não triviais armazenados.

Grandeza	FIR Hamming	IIR Butterworth
Ordem	44	8
Coeficientes na forma direta, sem a_0	45	17
Multiplicações por amostra, forma direta	45	17
Somas por amostra, forma direta	44	16
Rejeição mínima a partir de 150 Hz	43,33 dB	41,76 dB
Ripple até 80 Hz	0,078 dB	0,998 dB
Atraso de grupo na passagem	22 amostras	9,17 a 17,23 amostras
Alinhamento usado na métrica	22 amostras	9 amostras
Ganho de amplitude em 40 Hz	0,99906	0,999998
Ganho em 200 Hz	-63,00 dB	-66,41 dB
MSE sem alinhar	0,27232	1,72478
MSE alinhado	0,002253	0,008750
SNR alinhada	23,47 dB	17,57 dB
Fase linear	Sim	Não
Estabilidade causal	Estável	Estável

A entrada apresenta MSE de 0,18871 e SNR de 4,23 dB. As duas soluções melhoram a SNR alinhada e atendem às bandas especificadas. O FIR produz menor erro alinhado, enquanto o IIR exige menos operações e menor atraso na região de 40 Hz. A rejeição de 200 Hz é superior à rejeição mínima de banda, pois esta considera o pior ponto a partir de 150 Hz.

120 Interpretação e Discussão

O FIR tem coeficientes simétricos e atraso constante, enquanto o Butterworth causal tem menor ordem e atraso variável. A interferência de 200 Hz é atenuada segundo os ganhos medidos e o ganho de 40 Hz quantifica a preservação. MSE e SNR são calculados após excluir 500 amostras de transiente e alinhar a componente útil; também se informa o MSE sem alinhamento para não esconder a latência. O alinhamento do IIR usa fase em 40 Hz arredondada para amostras inteiras: é adequado à referência monofrequencial, mas não corrige a distorção de fase de um sinal de banda larga. Para este sensor com informação concentrada em 40 Hz, a escolha é o IIR por menor custo e latência; para pulsos cuja forma importa, o FIR merece preferência. A avaliação usa precisão de ponto flutuante, ruído estacionário e uma grade espectral finita; hardware com quantização dos coeficientes e sinais não estacionários exige nova validação.

121 Conclusões

As soluções FIR de ordem 44 e Butterworth de ordem 8 atendem à perda máxima de 1 dB e à rejeição mínima de 40 dB. A SNR aumenta de 4,23 dB para 23,47 dB no FIR e 17,57 dB no

IIR após o alinhamento descrito. A escolha do IIR prioriza custo e latência; a escolha do FIR prioriza fase linear e o menor erro obtido neste experimento. Nenhuma dessas conclusões dispensa a validação para outro sinal ou implementação numérica.

121.1 Conclusões da Parte 3

A filtragem exige especificações justificadas pelo sinal e verificação posterior. FIR oferece fase linear com maior comprimento; IIR oferece economia de ordem com atraso dependente da frequência. O mini projeto compara as mesmas tolerâncias e usa métricas de erro, SNR, ganho, rejeição e custo para justificar a escolha, sem omitir transientes e latência.

122 Referências bibliográficas

- SILVA, Moacy Pereira da. **Estudo Dirigido de Processamento Digital de Sinais — Parte 3**. IFPB, Campus Campina Grande, 2026.2. Guia fornecido em `docs/guia_parte_3.pdf`; fonte do roteiro e das investigações.
- OPPENHEIM, A. V.; SCHAFER, R. W. **Discrete-Time Signal Processing**. 3. ed. Pearson, 2010. Fundamentação de sinais, transformadas e filtros.
- PROAKIS, J. G.; MANOLAKIS, D. G. **Digital Signal Processing: Principles, Algorithms, and Applications**. 4. ed. Pearson, 2007.
- OPPENHEIM, A. V.; WILLSKY, A. S.; NAWAB, S. H. **Signals and Systems**. 2. ed. Prentice Hall, 1997. Sistemas contínuos e discretos, convergência e estabilidade.
- NUMPY. Discrete Fourier Transform. Convenções de DFT e frequências dos coeficientes. Acesso em: 8 nov. 2026.
- SCIPY. Signal processing. Rotinas de análise, discretização e projeto utilizadas nos notebooks. Acesso em: 8 nov. 2026.

Os guias definem o conteúdo; os livros são referências de aprofundamento. Os dados, exemplos numéricos e códigos deste estudo foram construídos para as simulações, sem aquisição experimental.